

Topic 6

Introduction to SAT

系統晶片驗證
SoC Verification

2025.04.02

Remember, we simplify verification problems as ---

1. Most of the bugs can be revealed by checking simple “**assertion** properties”.
2. Almost all the assertion properties can be remodeled (using auxiliary gates and/or FFs) as **AG** properties.
3. Most formal engines “prove” the $AG(p)$ properties by showing that there is NO counter-example for **EF(!p)**.
4. BDD is one important formal engine. It can compactly represent the circuit function/transition and compute the set of reachable states.

However, the problem with BDD is that it tries to compute all (or most) of the solutions at once, and thus may suffer from memory space explosion problem...

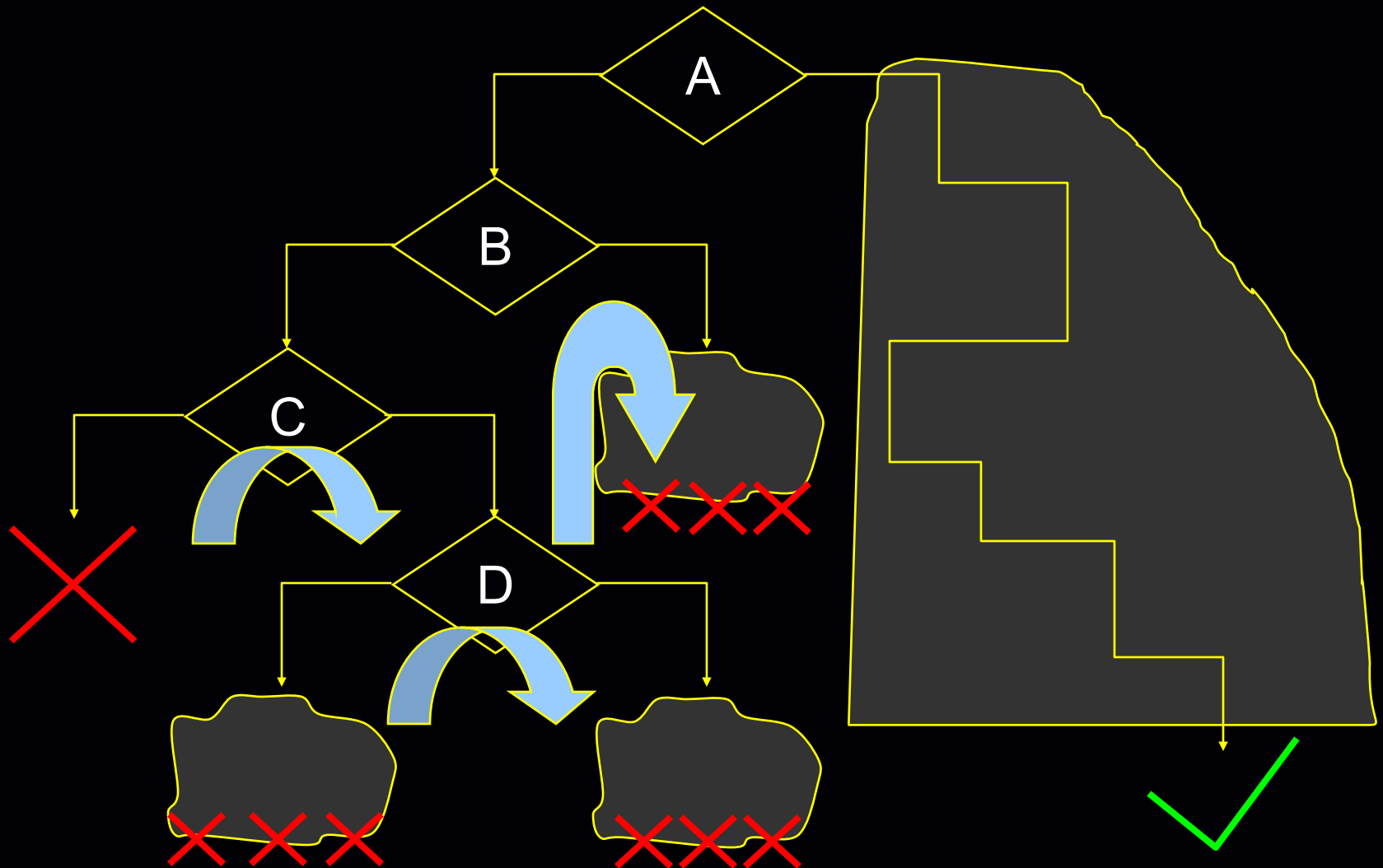
Can we trade some “time” for the “space”?

Think about the Human Search Process

(aka: search for !p)

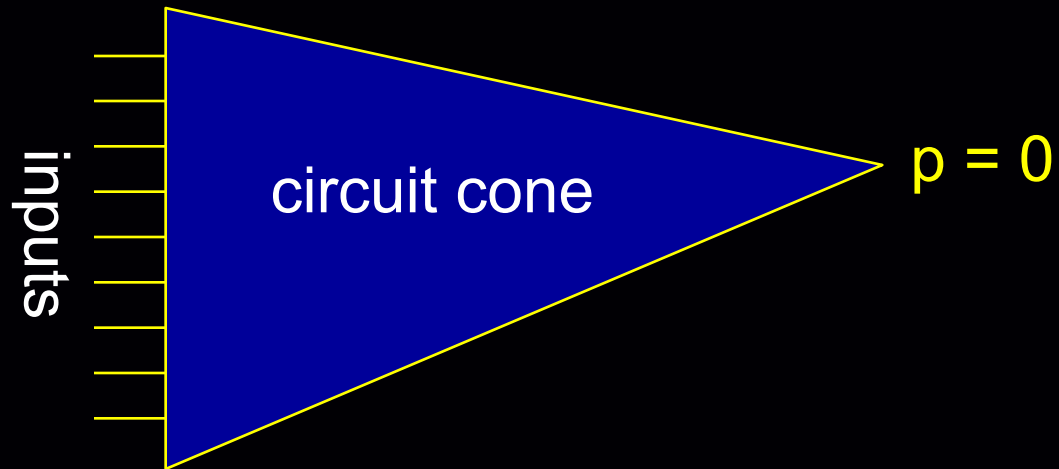
- ◆ When you search for something, you usually seek for one clue after another ---
 - Containing a keyword
 - Asking a question
 - By a direction, in a room, etc
- ◆ After some steps, if it is surely NOT there, you will reverse or revise some of the previous guesses/decisions and continue...

Something like --- A Decision Tree



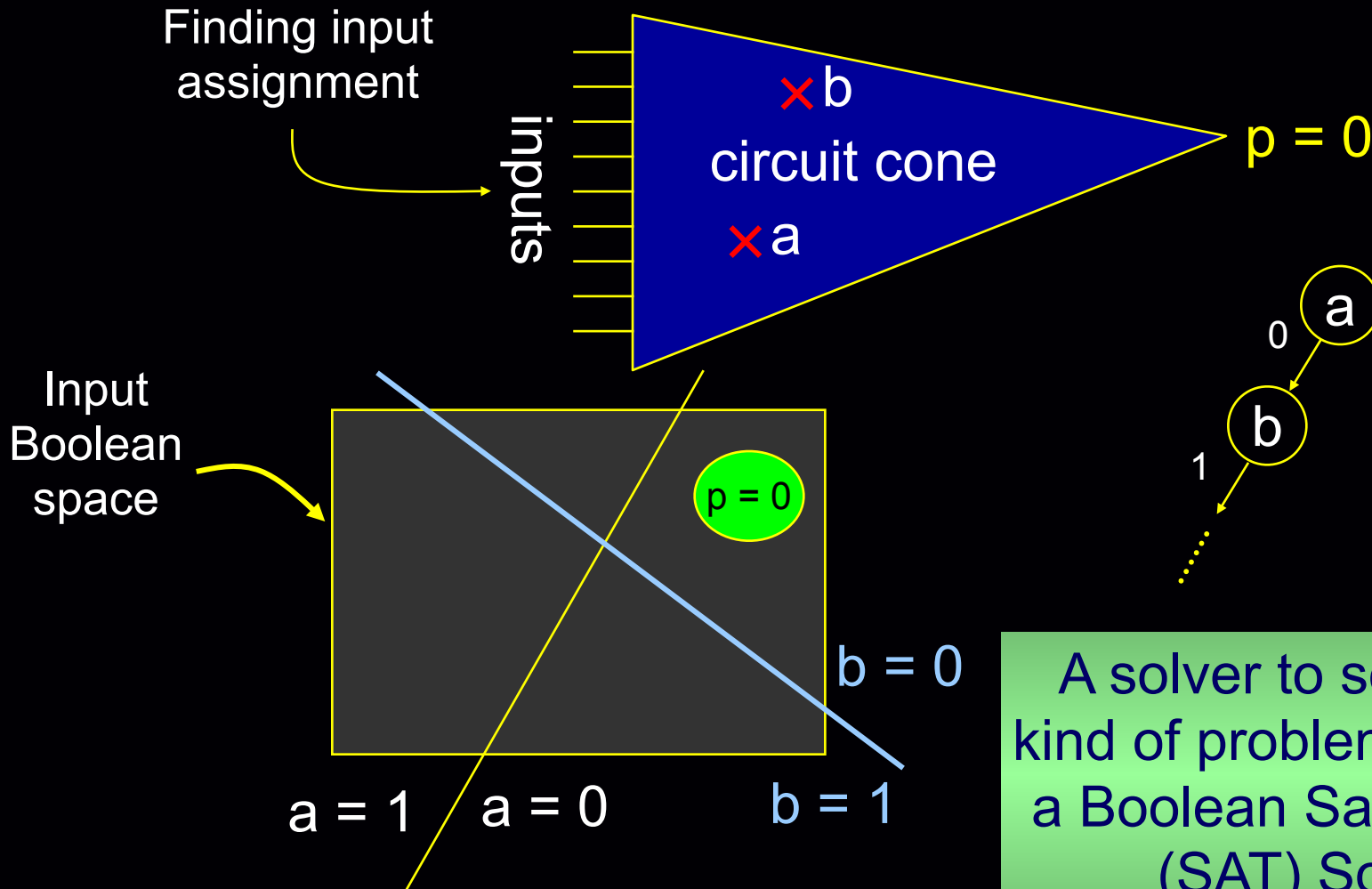
What about circuit properties?

Finding input
assignment to
violate p



- ◆ What are the “decisions”?
 - “Assigning input value one at a time” ?
 - Enumeration method: exponential complexity...
 - “Assigning values to internal signals” ?
 - Still exponential complexity...

Boolean Satisfiability Checking for Properties on Circuit



A solver to solve this kind of problem is called a Boolean Satisfiability (SAT) Solver

Introduction to Boolean Satisfiability (SAT)

A fundamental problem in computer science

- ◆ Given a Boolean network $F: B^n \rightarrow B$,
where $B = \{0, 1\}$, and
 n is the number of inputs $I = \{x_1, x_2, \dots, x_n\}$

- ◆ Boolean Satisfiability
→ Finding an input assignment

$$\mathbf{a}: \{x_1 = a_1, x_2 = a_2, \dots, x_n = a_n \mid a_i \in B\}$$

such that $F = 1$.

- ◆ Exponential complexity...?

Try this out --
SAT Game

Complexity of SAT solver

- ◆ Boolean Satisfiability (SAT) was the first proven NP-complete problem by Dr. S. Cook in 1971
 - Given n variables, the number of decisions can be as many as 2^n ...
 - If there is a non-deterministic machine, we can construct a polynomial-time algorithm that can guarantee to prove/disprove the SAT problem
- [Pitfall?] Unless there is a non-deterministic machine, we cannot construct a polynomial-time SAT algorithm (i.e. $P \neq NP$)

→ How can SAT be useable for million-gate designs?

Boolean Satisfiability Solvers

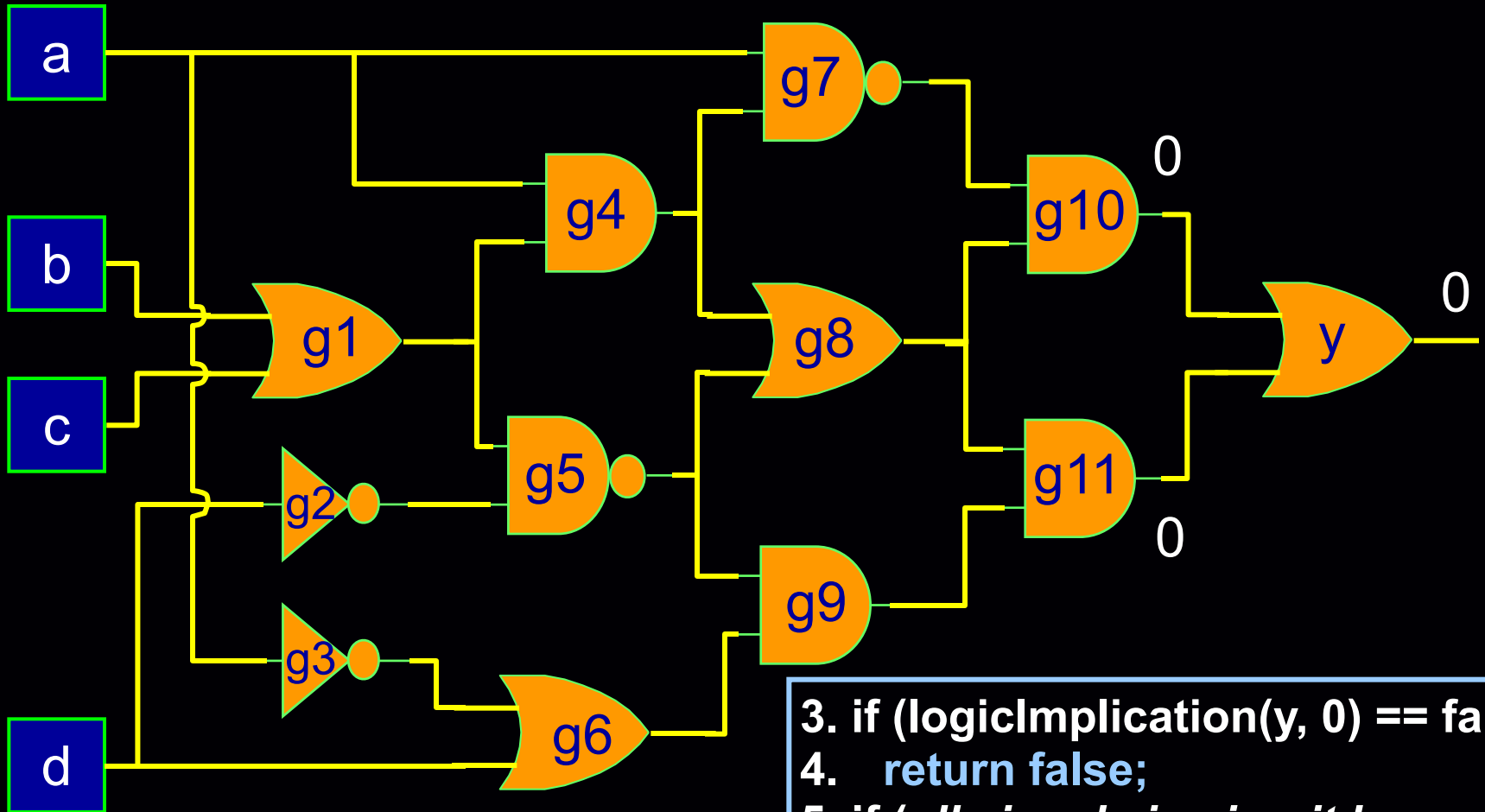
- ◆ Boolean SAT solvers have been very successful recent years in the verification area
 - More research / popular than BDDs
 - Applications
 - Equivalence checking, property checking, synthesis, etc
 - Applicable even on million-gate designs
 - For both combinational and sequential problems
- However, SAT is intrinsically a “combinational” (propositional) solver
- ◆ There are many advanced Boolean SAT algorithms
 - We will cover them gradually in the following lecture notes
- ◆ Many many SAT solvers
 - Grasp, SATO, zChaff, BerkMin, miniSat, glucose, (P)lingeling, MapleXXX, kissat, SBVA,... etc.
 - <http://www.satcompetition.org/>

A Very Basic SAT Algorithm

```
1. bool sat(Gate g, Value v)
2. {
3.     if (logicImplication(g, v) == false) // conflict
4.         return false;
5.     if (all signals in circuit have been implied)
6.         return true;
7.     pick an unassigned signal s
8.     if (sat(s, V0) == true) // found a solution
9.         return true;
10.    backtrack(s);
11.    if (sat(s, ~V0) == true) // found a solution
12.        return true;
13.    backtrack(s);
14.    return false; // no solution
15.}
```

Proving $\text{always}(y == 1)$

$\text{sat}(y, 0)$



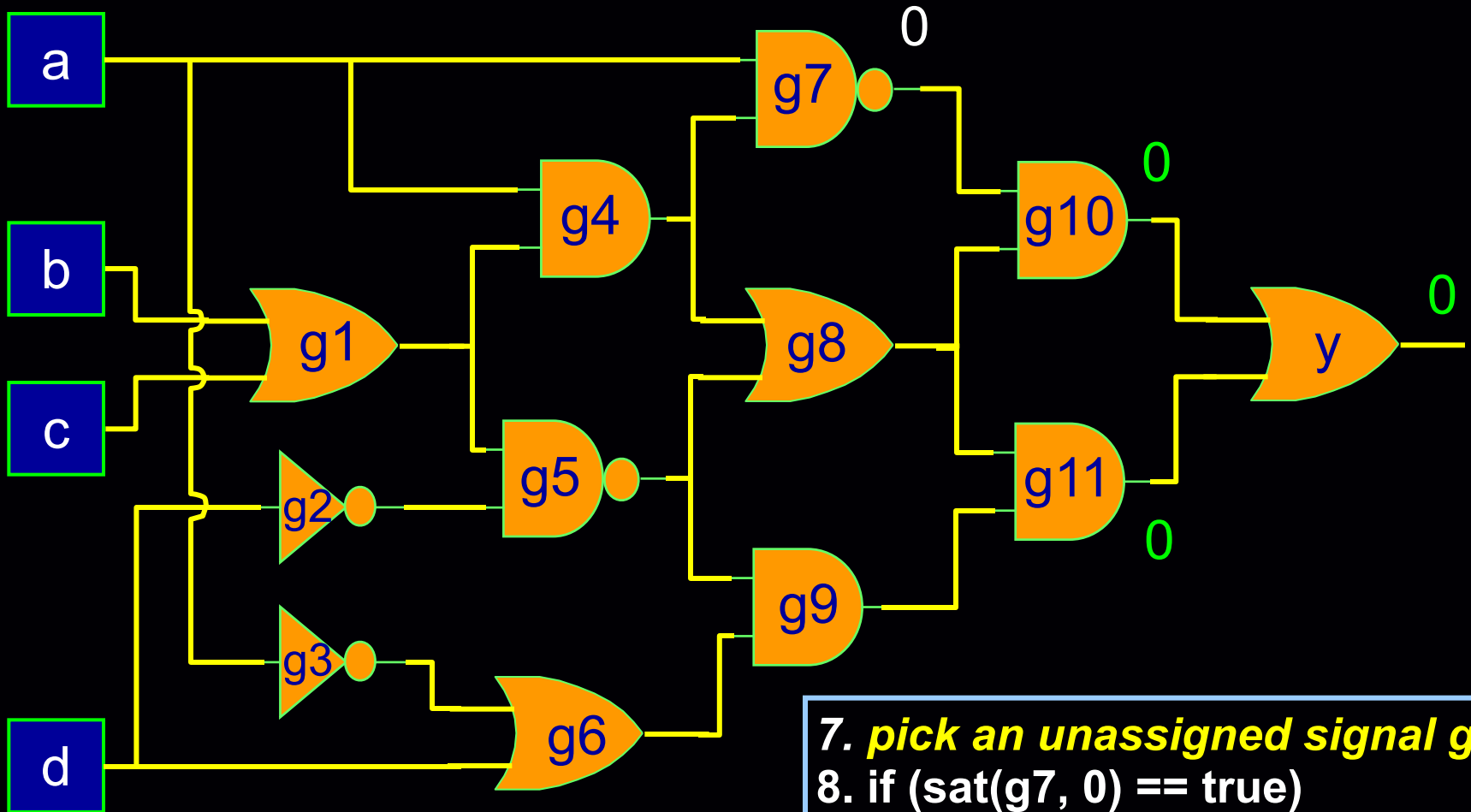
```
3. if (logicImplication(y, 0) == false)
4.   return false;
5. if (all signals in circuit have
6.   been implied) return true;
```

A Very Basic SAT Algorithm

```
1. bool sat(Gate g, Value v)
2. {
3.     if (logicImplication(g, v) == false) // conflict
4.         return false;
5.     if (all signals in circuit have been implied)
6.         return true;
7.     pick an unassigned signal s
8.     if (sat(s, V0) == true) // found a solution
9.         return true;
10.    backtrack(s);
11.    if (sat(s, ~V0) == true) // found a solution
12.        return true;
13.    backtrack(s);
14.    return false;
15.}
```

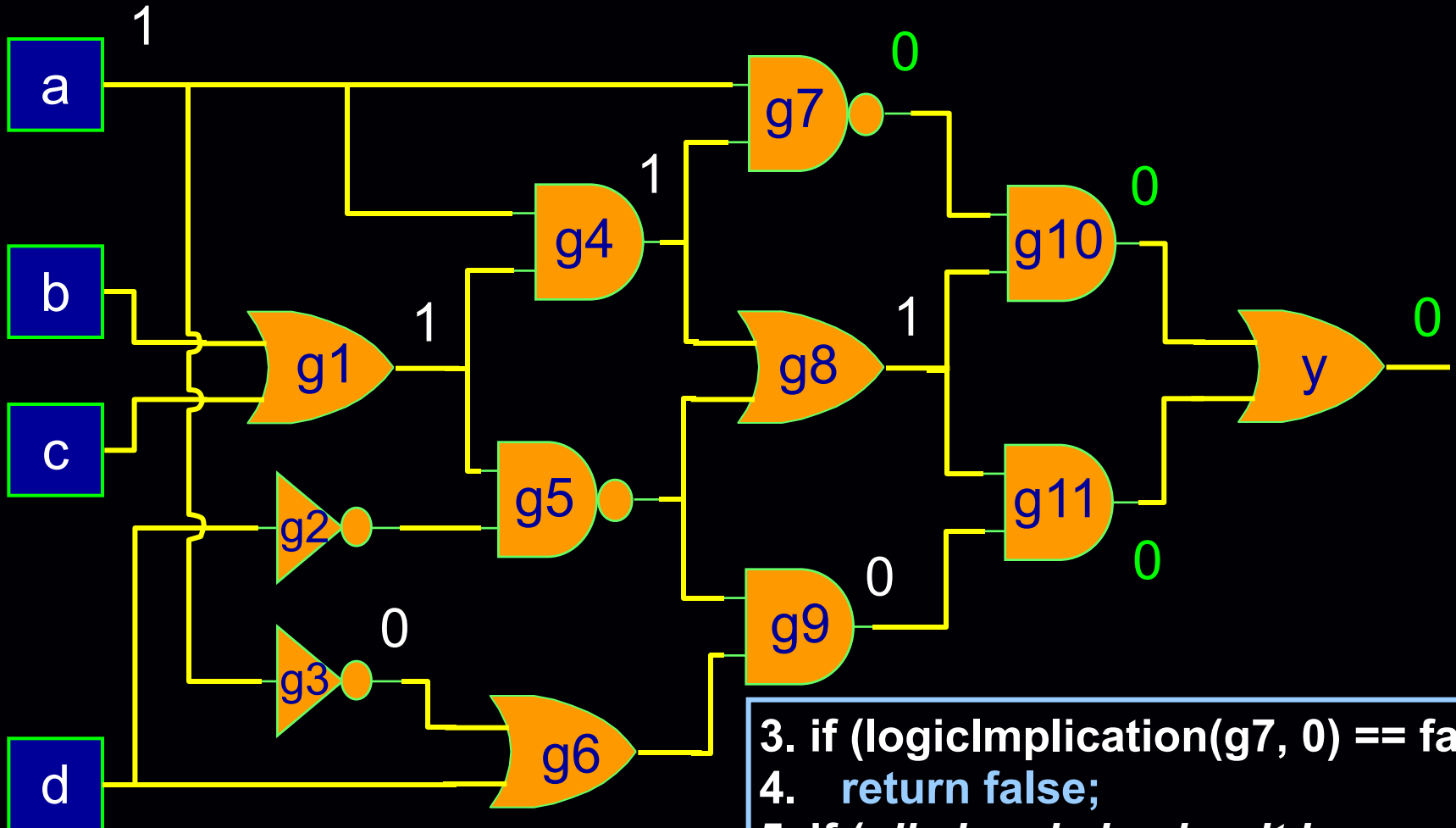
Solve: sat (y, 0)

sat (g7, 0)



7. ***pick an unassigned signal g7***
8. ***if (sat(g7, 0) == true)***
9. ***return true;***

Recursion: sat (g7, 0)



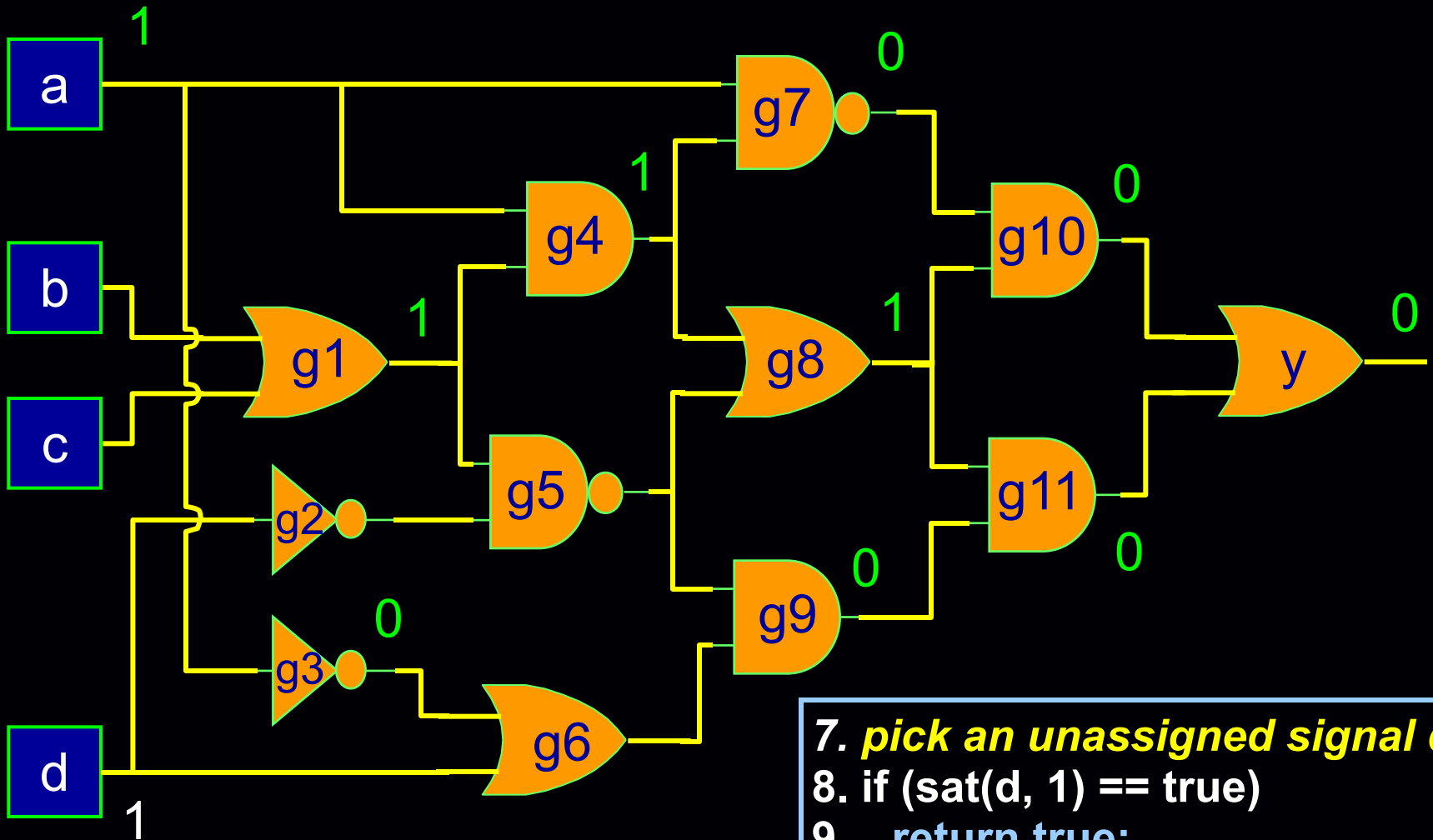
```

3. if (logicImplication(g7, 0) == false)
4.   return false;
5. if (all signals in circuit have
6.   been implied) return true;

```

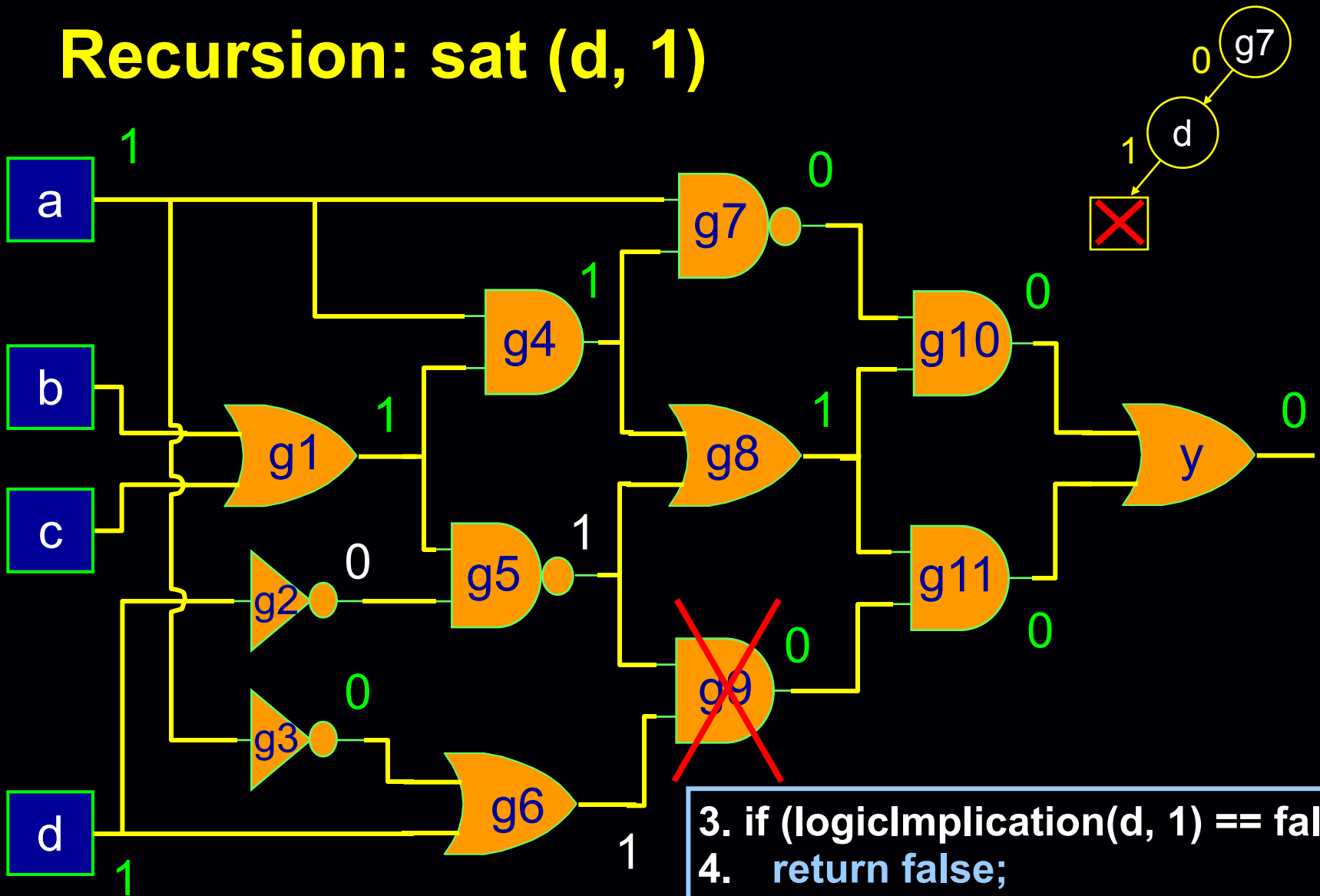
Recursion: sat (g7, 0)

sat (d, 1)



```
7. pick an unassigned signal d  
8. if (sat(d, 1) == true)  
9.   return true;
```

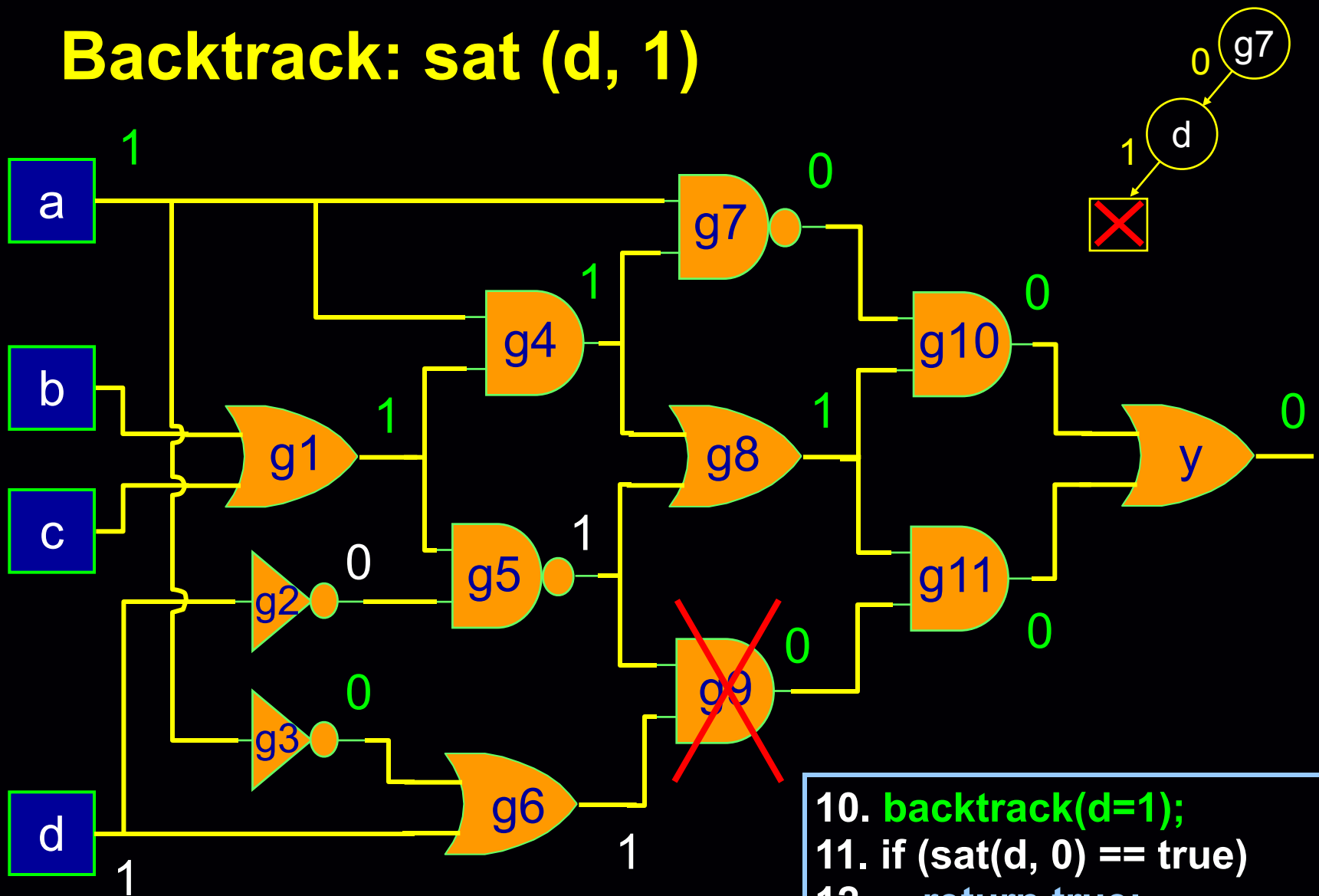

Recursion: sat (d, 1)



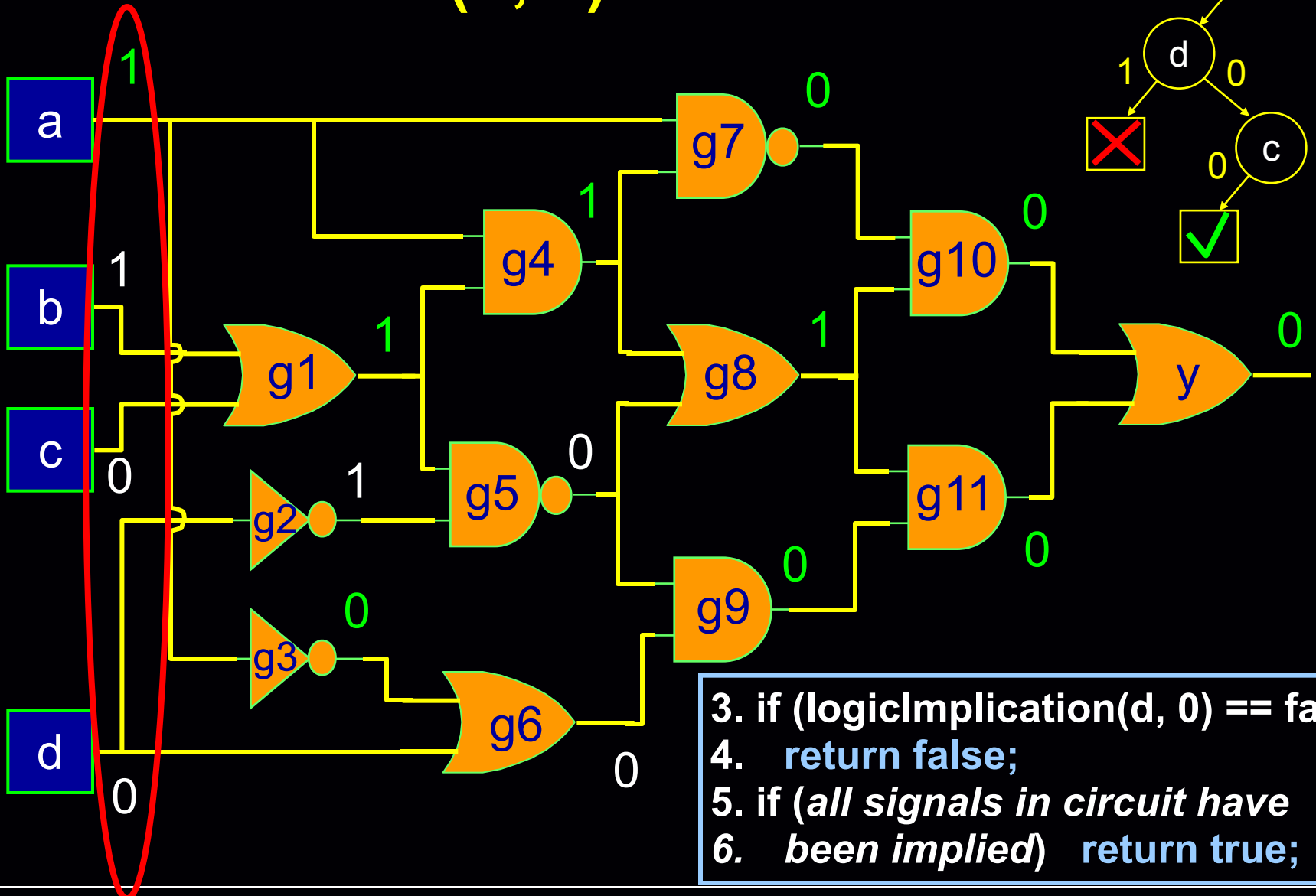
A Very Basic SAT Algorithm

```
1. bool sat(Gate g, Value v)
2. {
3.     if (logicImplication(g, v) == false) // conflict
4.         return false;
5.     if (all signals in circuit have been implied)
6.         return true;
7.     pick an unassigned signal s
8.     if (sat(s, V0) == true) // found a solution
9.         return true;
10.    backtrack(s);
11.    if (sat(s, ~V0) == true) // found a solution
12.        return true;
13.    backtrack(s);
14.    return false;
15.}
```

Backtrack: sat (d, 1)



Branch: sat (d, 0)



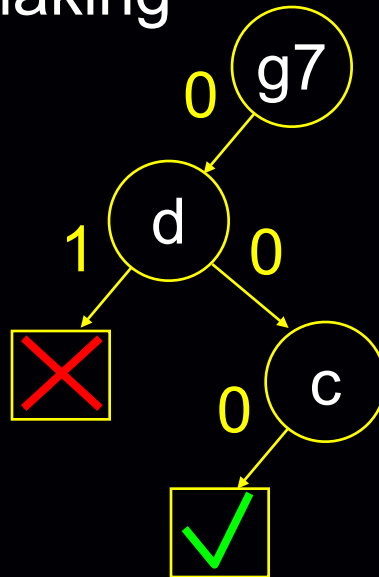
```

3. if (logicImplication(d, 0) == false)
4.   return false;
5. if (all signals in circuit have
6.   been implied) return true;

```

What are the key steps in a SAT algorithm?

- ◆ Decision making



- ◆ Logic implication (Boolean Constraint Propagation, BCP)
- ◆ What else?

In the previous “very basic SAT algorithm”, we make decisions and apply logic implications on the circuit.

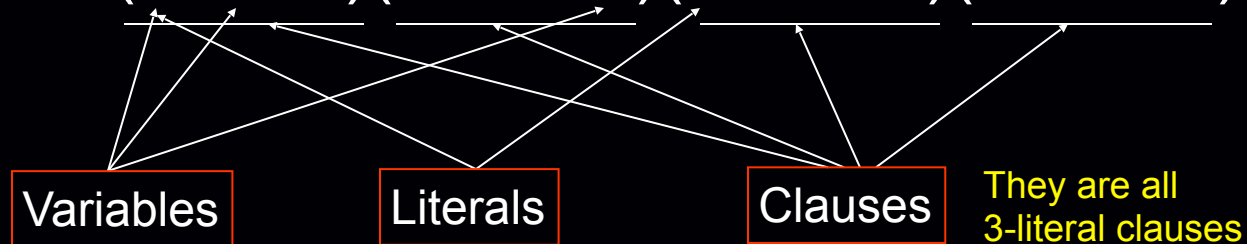
However, in practice, most SAT solvers are implemented on a “Conjunctive Normal Form (CNF)” data structure.

Types of Boolean Satisfiability Solvers

1. Conjunctive Normal Form (CNF) Based

- Boolean function is represented as a CNF (i.e. Product of Sum, POS format)

- e.g. $(a+b+c)(a'+b'+c)(a'+b+c')(a+b'+c')$



- To be satisfied, all the clauses should be '1'

2. Circuit-Based

- Boolean function is represented as a circuit netlist
- SAT algorithm is directly operated on the netlist

CNF vs. Circuit SAT

- ◆ Although CNF and circuit SAT solvers look quite different, their algorithms can be very similar
 - ◆ CNF SAT
 - Simpler data structure; easier to implement
 - ◆ Circuit SAT
 - Structural information; extensible to word-level
- In the following slides, we will focus on the easier-to-implement solver, CNF SAT, first

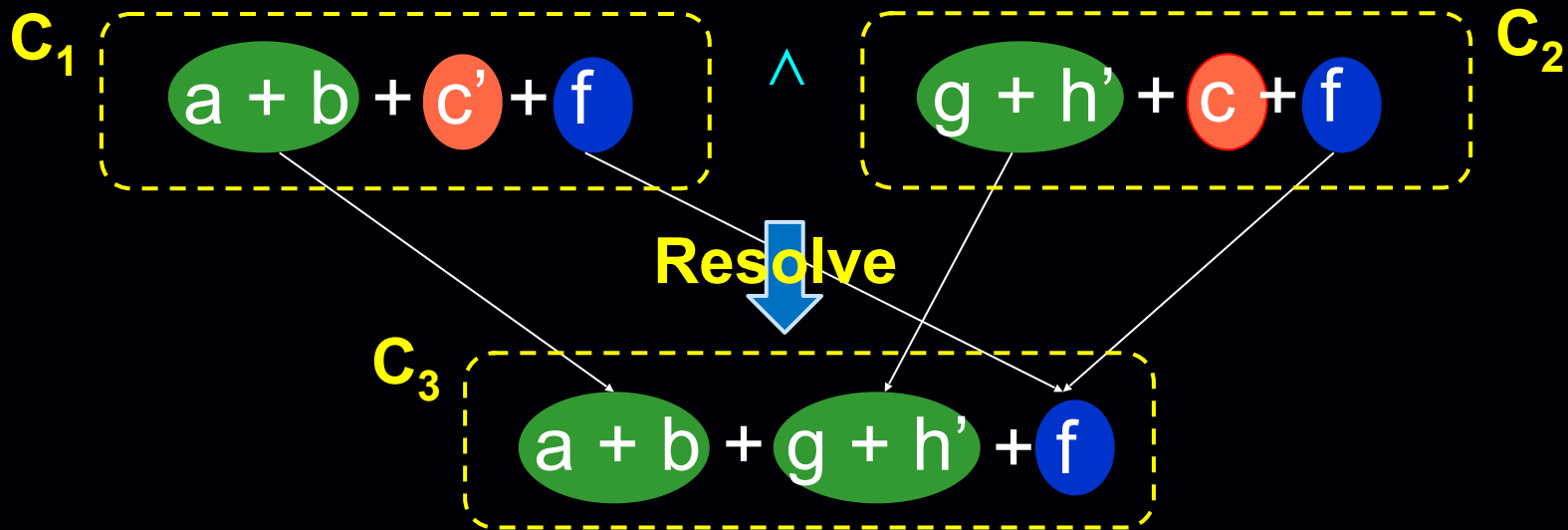
CNF-Based SAT Algorithm

1. Davis, Putnam, 1960
 - Explicit resolution based
 - May explode in memory
2. Davis, Logemann, Loveland, (DLL) 1962
 - Search based.
 - Most successful, basis for almost all modern SAT solvers
 - Learning and non-chronological backtracking, 1996
3. Stålmarcks algorithm, 1980s
 - Proprietary algorithm. Patented.
 - Commercial versions available
4. Stochastic Methods, 1992
 - Unable to prove unsatisfiability, but may find solutions for a satisfying problem quickly.
 - Local search and hill climbing

Source: Prof. Sharad Malik, "The Quest for Efficient Boolean Satisfiability Solvers"

Resolution

- ◆ Resolution of a pair of clauses with exactly ONE incompatible variable
 - Two clauses are said to have distance 1



- $C_1 \wedge C_2 \rightarrow C_3$ or $C_3 \rightarrow C_1 \wedge C_2$?
- Existential quantification?

Source: Prof. Sharad Malik, "The Quest for Efficient Boolean Satisfiability Solvers"

Davis Putnam Algorithm

M. Davis, H. Putnam, "A computing procedure for quantification theory", *J. of ACM*, Vol. 7, pp. 201-214, 1960 (360 citations in citeseer)

- ♦ Existential abstraction using resolution
- ♦ Iteratively select a variable for resolution till no more variables are left.

$(a + \textcircled{b} + c) (\textcircled{b} + c' + f) (\textcircled{b'} + e)$

$(a + \textcircled{c} + e) (\textcircled{c'} + e + f)$

$(a + e + f)$

SAT

Sol: $\{a=1, e=1, f=1\}$

$(a + \textcircled{b}) (a + \textcircled{b'}) (a' + c) (a' + c')$

$(\textcircled{a}) (\textcircled{a'} + c) (\textcircled{a'} + c')$

$(\textcircled{c}) (\textcircled{c'})$

$()$

UNSAT

Potential memory explosion problem!

Source: Prof. Sharad Malik, "The Quest for Efficient Boolean Satisfiability Solvers"

Boolean Satisfiability (SAT) Algorithm

1. Davis, Putnam, 1960
 - Explicit resolution based
 - May explode in memory
2. Davis, (Putnam), Logemann, Loveland, (D(P)LL) 1962
 - Search based.
 - Most successful, basis for almost all modern SAT solvers
 - Learning and non-chronological backtracking, 1996
3. Stålmarcks algorithm, 1980s
 - Proprietary algorithm. Patented.
 - Commercial versions available
4. Stochastic Methods, 1992
 - Unable to prove unsatisfiability, but may find solutions for a satisfying problem quickly.
 - Local search and hill climbing

Basic DLL Procedure - DFS

$$(a' + b + c)$$

$$(a + c + d)$$

$$(a + c + d')$$

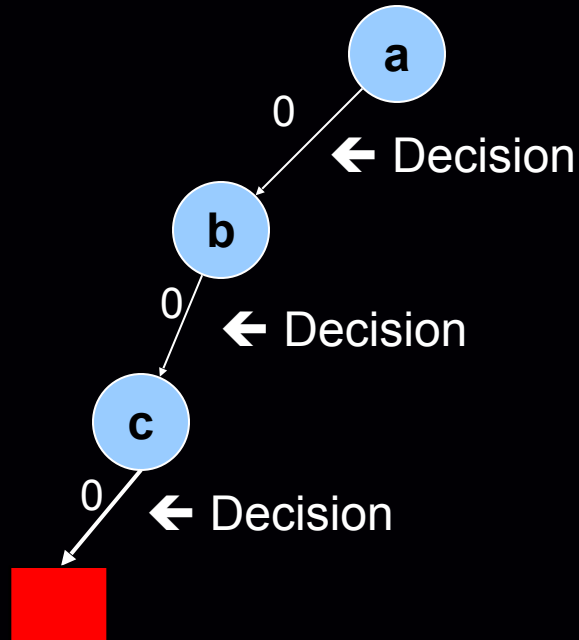
$$(a + c' + d)$$

$$(a + c' + d')$$

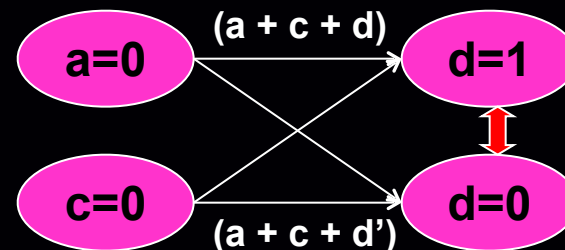
$$(b' + c' + d)$$

$$(a' + b + c')$$

$$(a' + b' + c)$$



Implication Graph



Conflict!

Basic DLL Procedure - DFS

$(a' + b + c)$

$(a + c + d)$

$(a + c + d')$

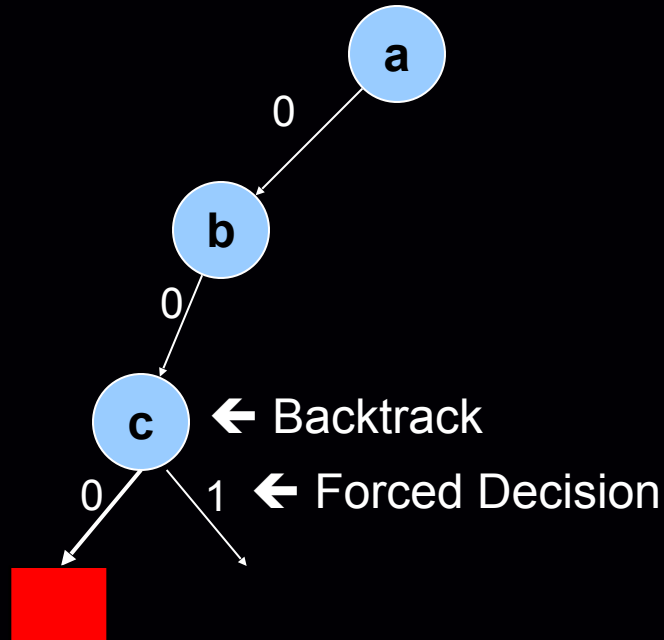
$(a + c' + d)$

$(a + c' + d')$

$(b' + c' + d)$

$(a' + b + c')$

$(a' + b' + c)$



Basic DLL Procedure - DFS

$$(a' + b + c)$$

$$(a + c + d)$$

$$(a + c + d')$$

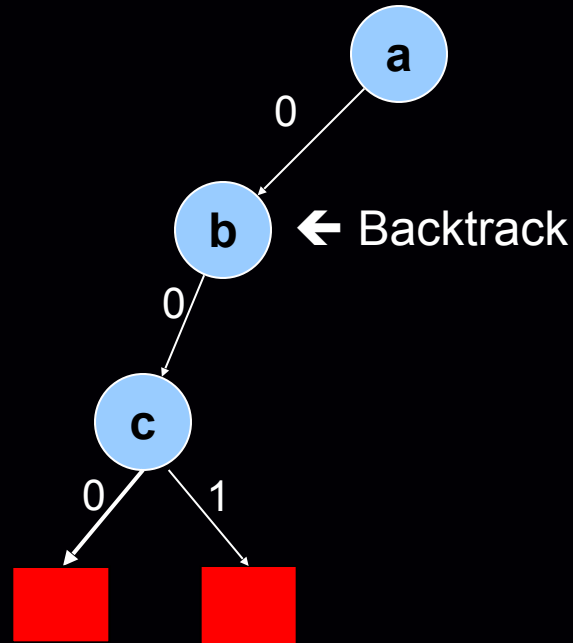
$$(a + c' + d)$$

$$(a + c' + d')$$

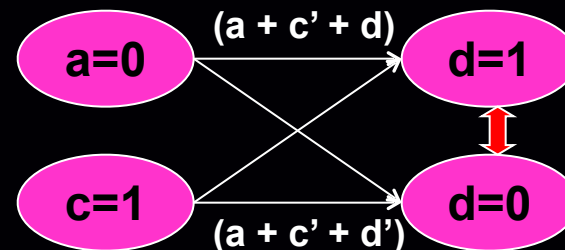
$$(b' + c' + d)$$

$$(a' + b + c')$$

$$(a' + b' + c)$$



Implication Graph



Conflict!

Basic DLL Procedure - DFS

$$(a' + b + c)$$

$$(a + c + d)$$

$$(a + c + d')$$

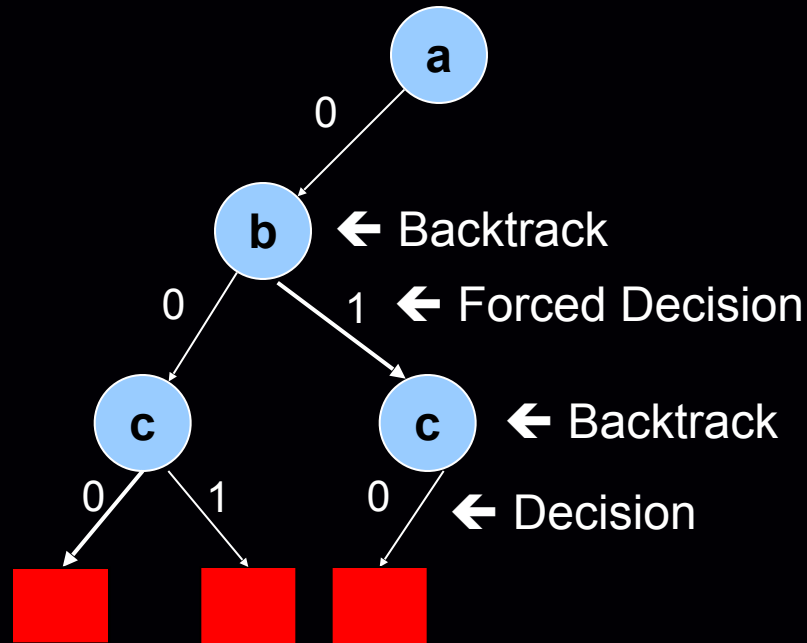
$$(a + c' + d)$$

$$(a + c' + d')$$

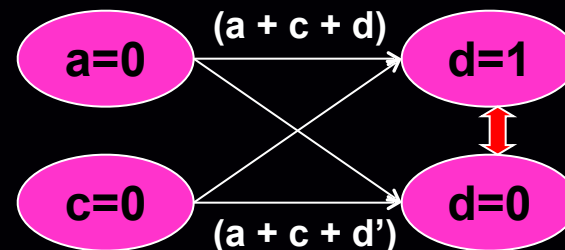
$$(b' + c' + d)$$

$$(a' + b + c')$$

$$(a' + b' + c)$$



Implication Graph



Conflict!

Basic DLL Procedure - DFS

$$(a' + b + c)$$

$$(a + c + d)$$

$$(a + c + d')$$

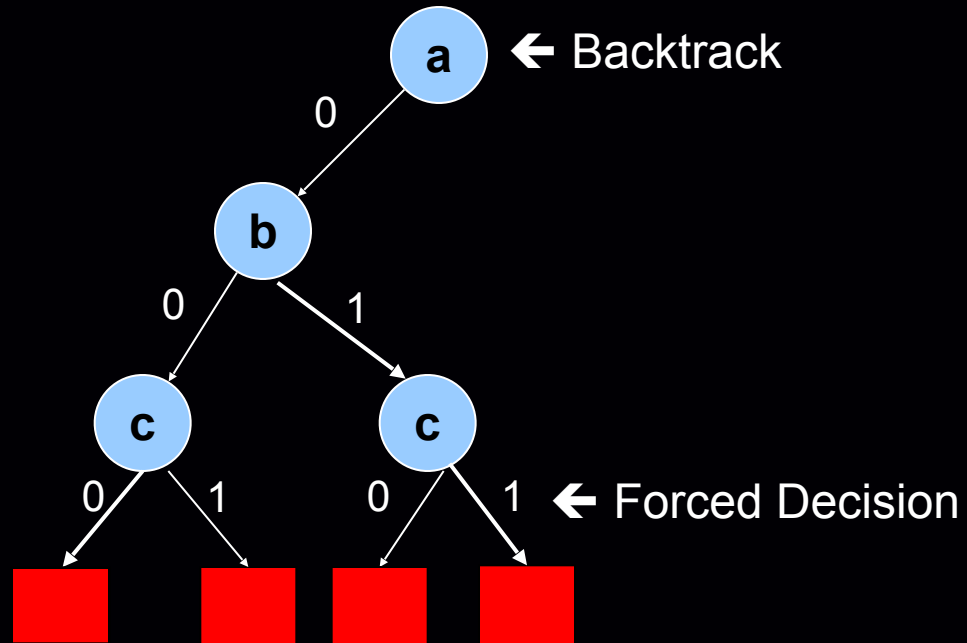
$$(a + c' + d)$$

$$(a + c' + d')$$

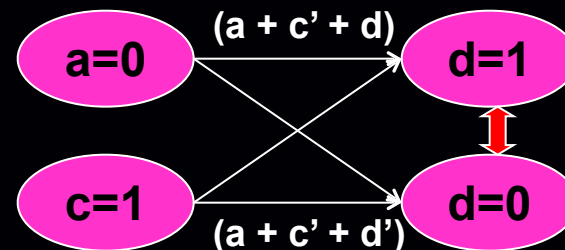
$$(b' + c' + d)$$

$$(a' + b + c')$$

$$(a' + b' + c)$$



Implication Graph



Conflict!

Basic DLL Procedure - DFS

$$(a' + b + c)$$

$$(a + c + d)$$

$$(a + c + d')$$

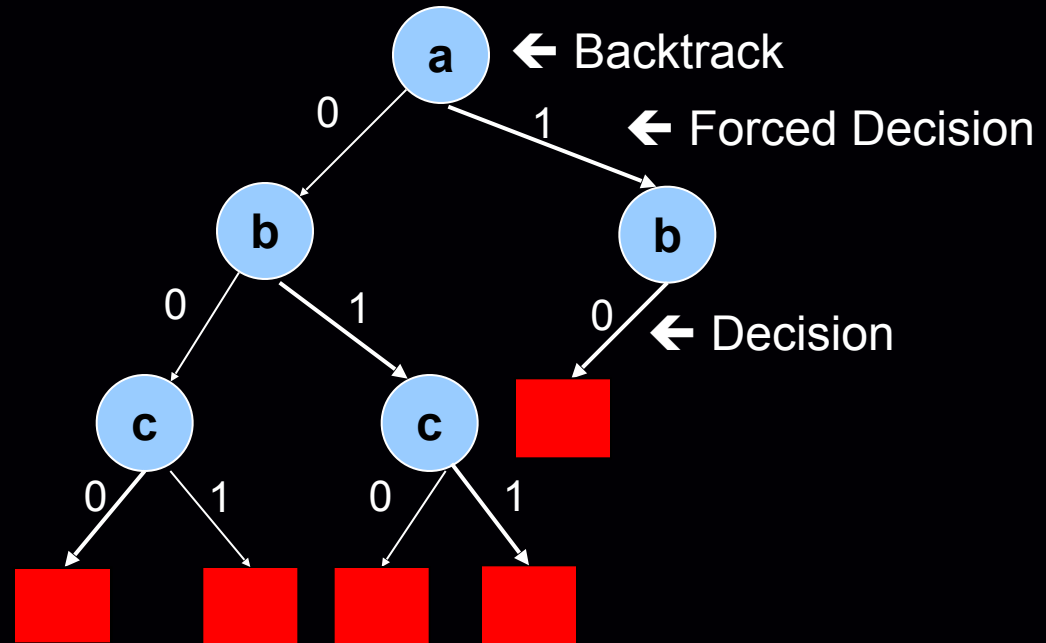
$$(a + c' + d)$$

$$(a + c' + d')$$

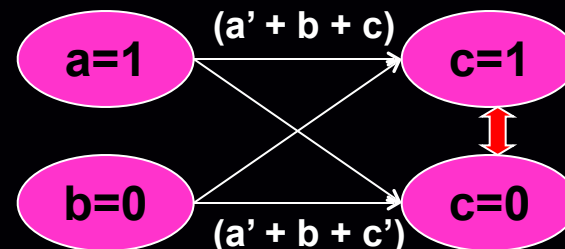
$$(b' + c' + d)$$

$$(a' + b + c')$$

$$(a' + b' + c)$$



Implication Graph



Conflict!

Basic DLL Procedure - DFS

$(a' + b + c)$

$(a + c + d)$

$(a + c + d')$

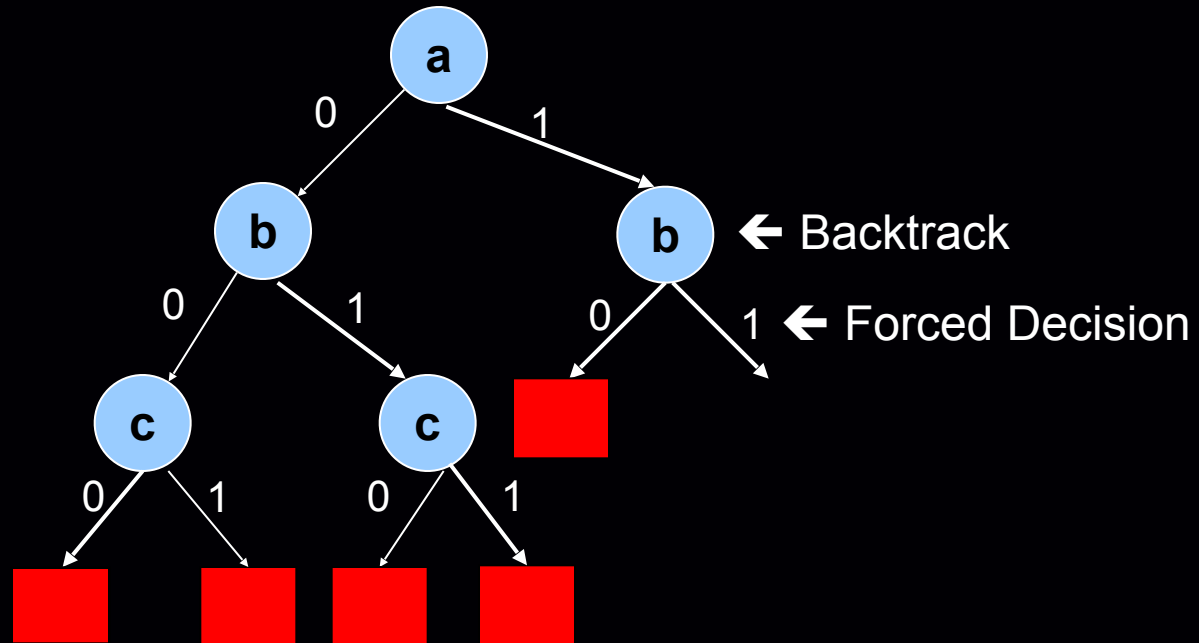
$(a + c' + d)$

$(a + c' + d')$

$(b' + c' + d)$

$(a' + b + c')$

$(a' + b' + c)$



Basic DLL Procedure - DFS

$$(a' + b + c)$$

$$(a + c + d)$$

$$(a + c + d')$$

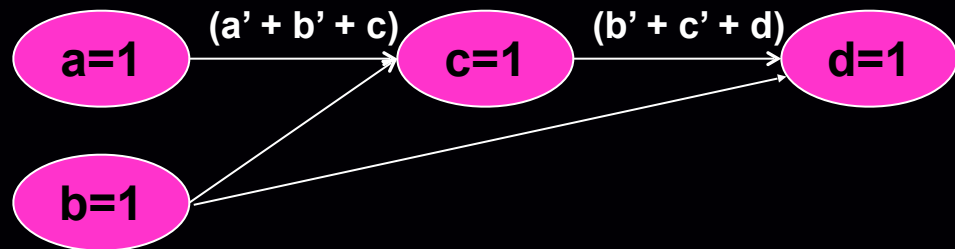
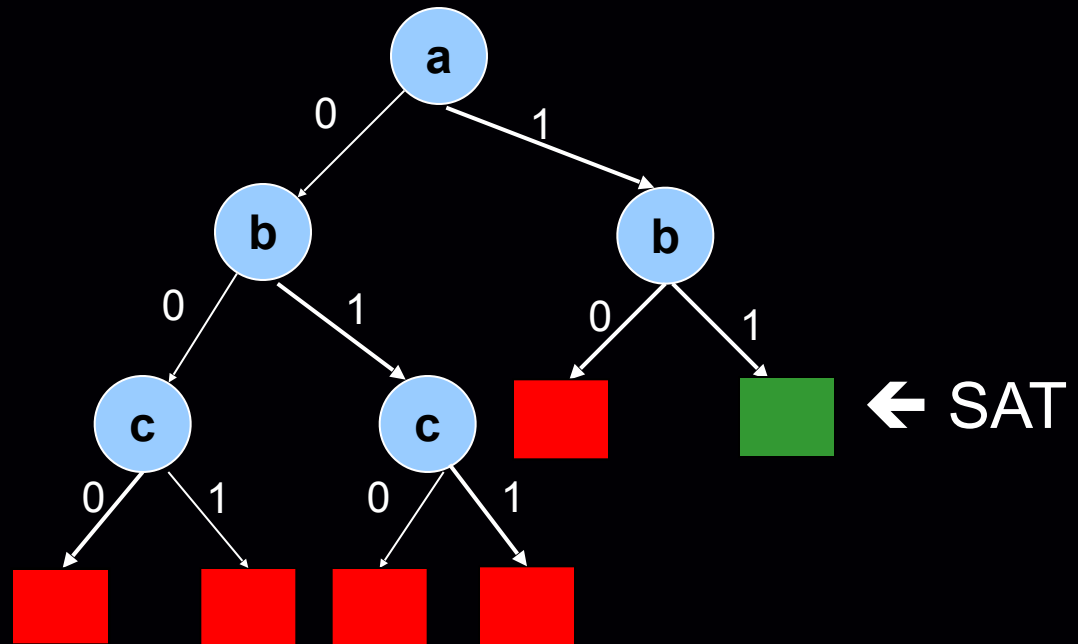
$$(a + c' + d)$$

$$(a + c' + d')$$

$$(b' + c' + d)$$

$$(a' + b + c')$$

$$(a' + b' + c)$$



Potentially exponential
complexity!!

Did you see any unnecessary
work?

SAT Improvements

1. Conflict-driven learning

- Once we encounter a conflict
 - Figure out the cause(s) of this conflict and prevent to see this conflict again!!

Conflict-Driven Learning

$(a' + b + c)$

$(a + c + d)$

$(a + c + d')$

$(a + c' + d)$

$(a + c' + d')$

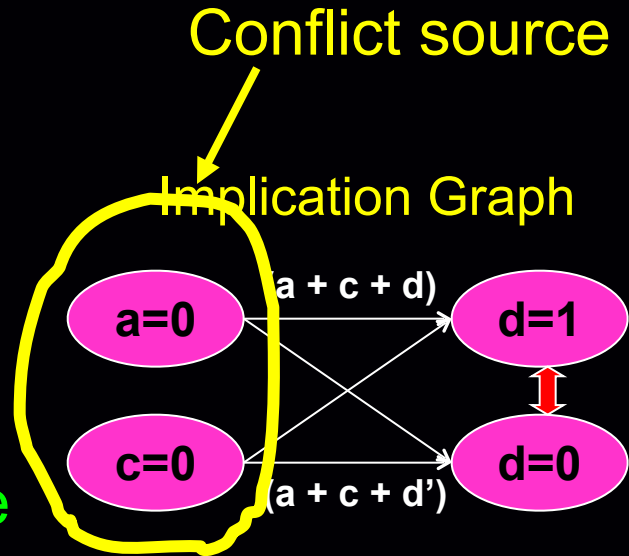
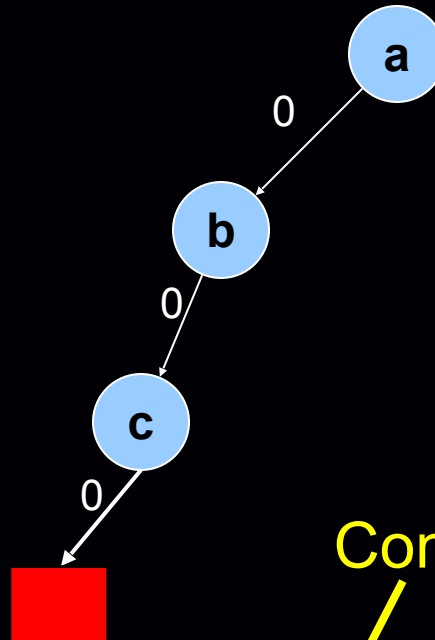
$(b' + c' + d)$

$(a' + b + c')$

$(a' + b' + c)$

$(a + c)$

Learned clause



SAT Improvements

2. Non-chronological backtracking

- Since we get a learned clause from the conflict analysis...
 - Instead of backtracking 1 decision at a time, backtrack to the “next-to-the-last” variable in the learned clause

Non-Chronological Backtracking

$(a' + b + c)$

$(a + c + d)$

$(a + c + d')$

$(a + c' + d)$

$(a + c' + d')$

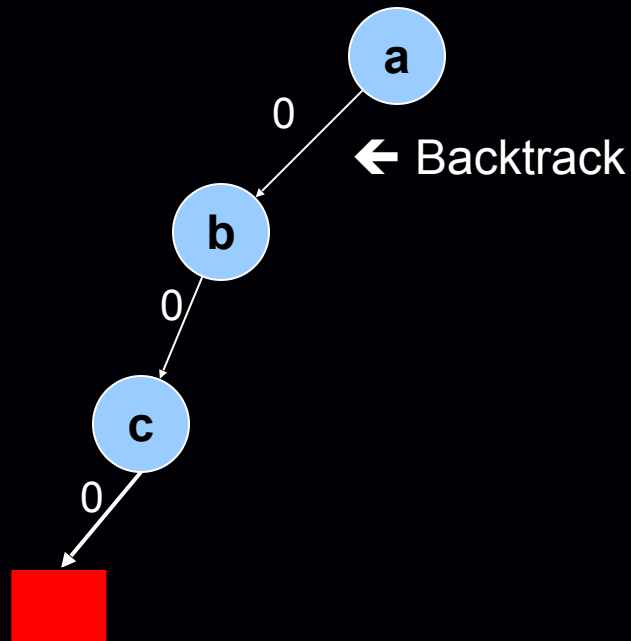
$(b' + c' + d)$

$(a' + b + c')$

$(a' + b' + c)$

$(a + c)$

Learned clause



- 'a' is the next-to-the-last variable in the learned clause
- Backtrack $c = 0 \ \&\& \ b = 0$

Deduced Implication from Learned Clause

$$(a' + b + c)$$

$$(a + c + d)$$

$$(a + c + d')$$

$$(a + c' + d)$$

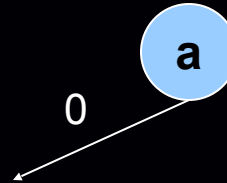
$$(a + c' + d')$$

$$(b' + c' + d)$$

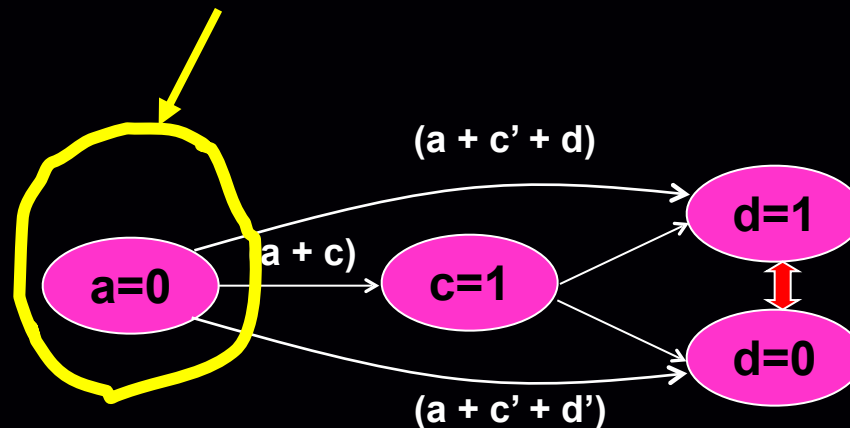
$$(a' + b + c')$$

$$(a' + b' + c)$$

$$(a + c)$$



Conflict source



Conflict!

Deduced Implication from Learned Clause

$(a' + b + c)$

$(a + c + d)$

$(a + c + d')$

$(a + c' + d)$

$(a + c' + d')$

$(b' + c' + d)$

$(a' + b + c')$

$(a' + b' + c)$

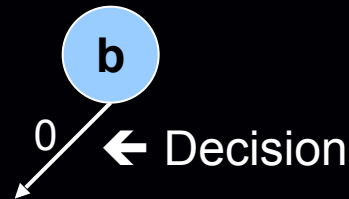
$(a + c)$

(a) Learned clause

- Since there is only one variable in the learned clause
→ No one is the next-to-the-last variable
- Backtrack all decisions
→ $(a = 1)$ is asserted

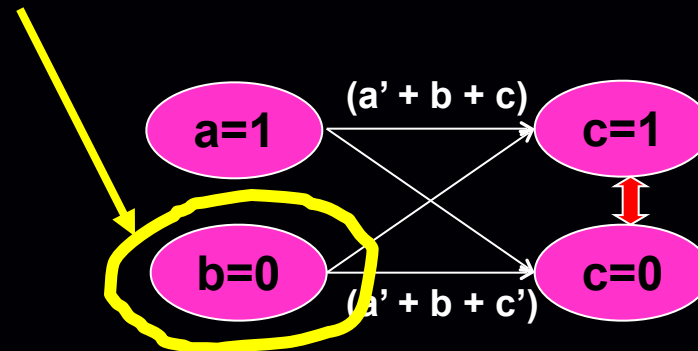
Deduced Implication from Learned Clause

$(a' + b + c)$
$(a + c + d)$
$(a + c + d')$
$(a + c' + d)$
$(a + c' + d')$
$(b' + c' + d)$



$(a' + b + c')$
$(a' + b' + c)$
$(a + c)$
(a)

Conflict source



Conflict!

Deduced Implication from Learned Clause

$$(a' + b + c)$$

$$(a + c + d)$$

$$(a + c + d')$$

$$(a + c' + d)$$

$$(a + c' + d')$$

$$(b' + c' + d)$$

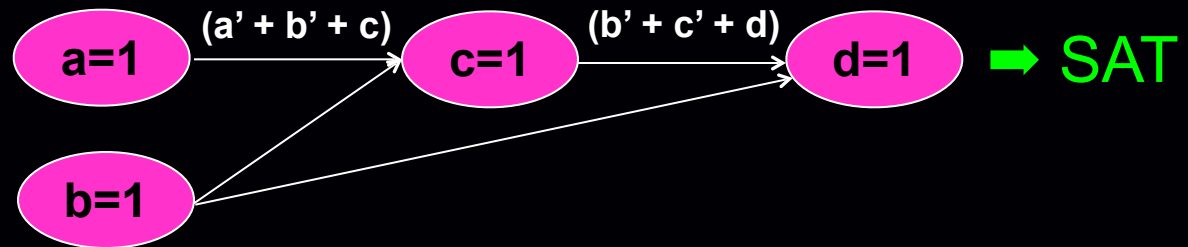
$$(a' + b + c')$$

$$(a' + b' + c)$$

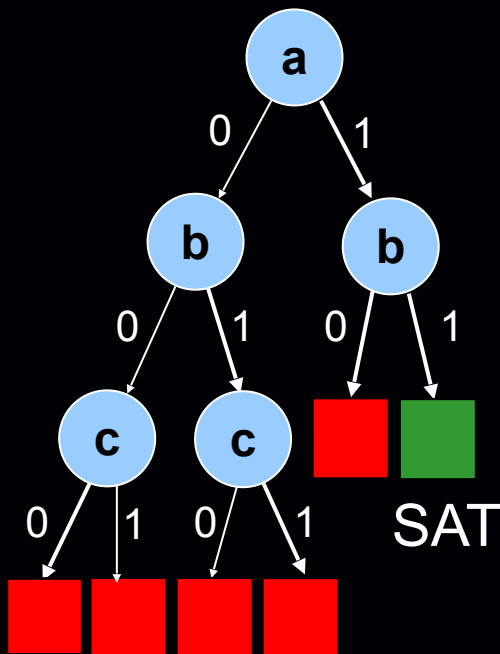
$$(a + c)$$

$$(a)(b)$$

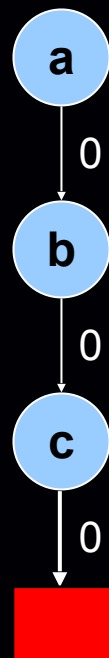
Learned clause



Without vs. With Conflict-Driven Learning

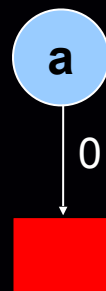


Without
conflict-driven
learning

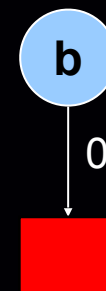


Conflict !!

(a + c) is learned



Backtrack to 'a'
(c = 1) is implied
Conflict !!
(a) is learned



Backtrack ALL
(a = 1) is asserted
Decision (b = 0)
Conflict !!
(b) is learned

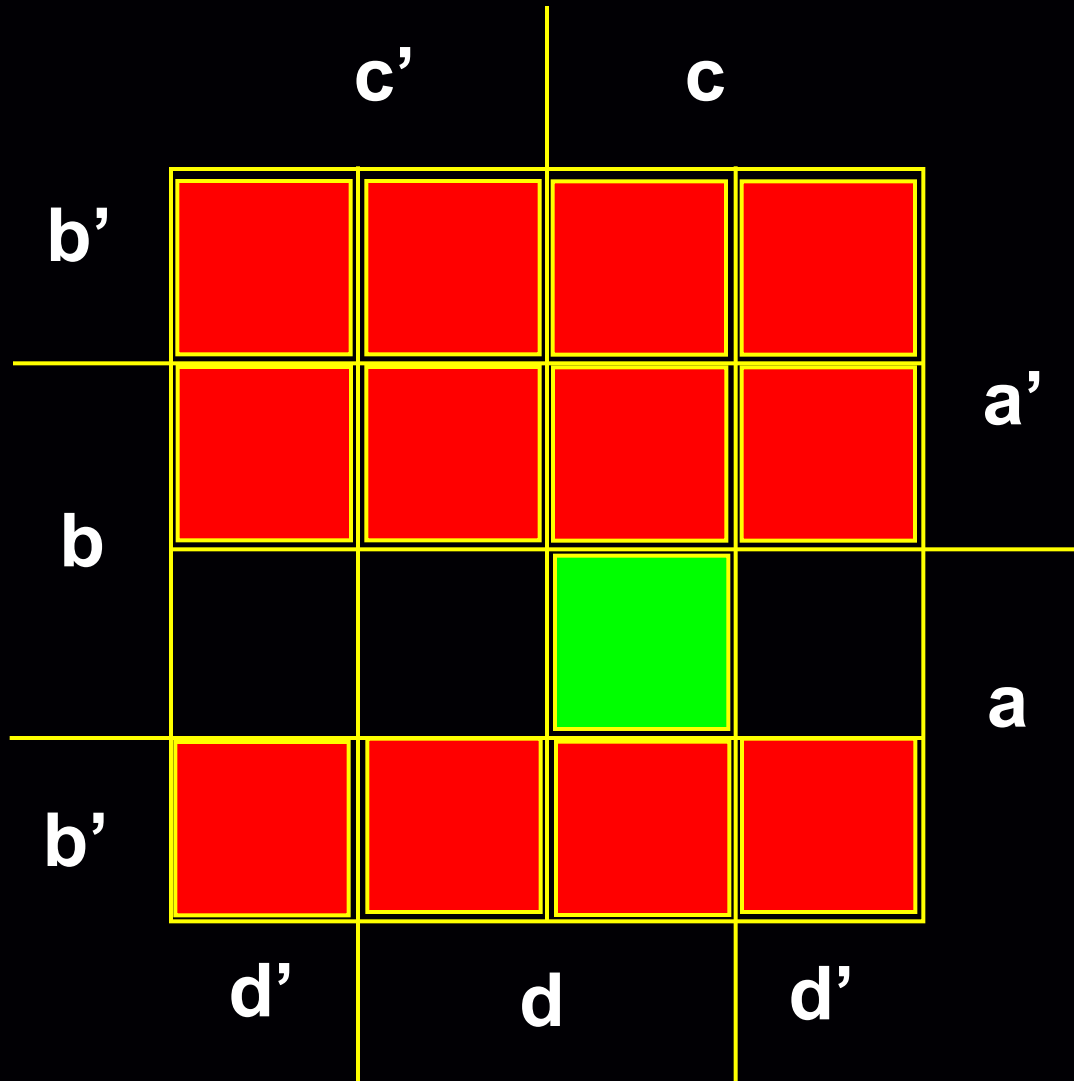


SAT

Backtrack ALL
(b = 1) is asserted
(c = 1) is implied
(d = 1) is implied

With conflict-driven learning

Conflict-Driven Learning = Constraint Refinement Process



Decision: $a = 0$

Decision: $b = 0$

Decision: $c = 0$

conflict!!

Learned: $(a + c)$

Backtrack: $c = 0, b = 0$

Implied: $c = 1$

conflict!!

Learned: (a)

Implied: $a = 1$

Decision: $b = 0$

conflict!!

Learned: (b)

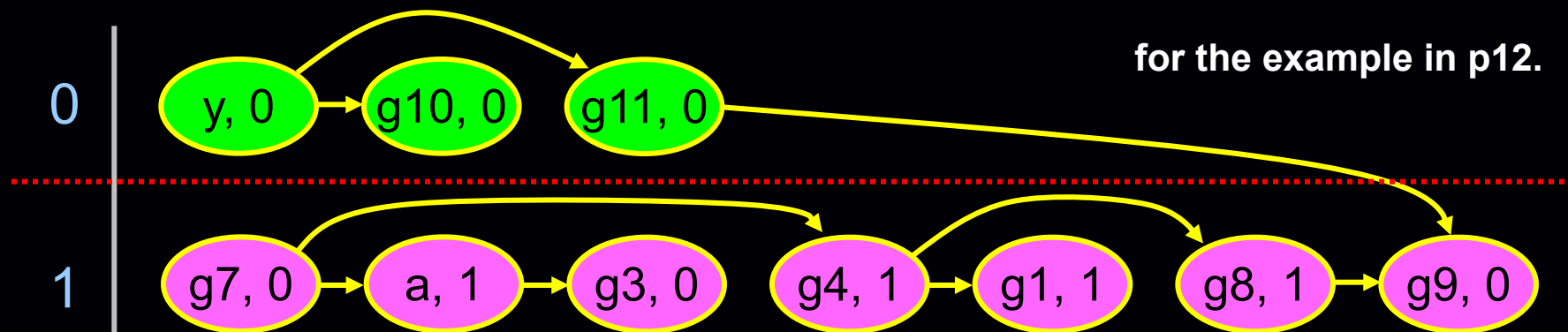
Implied: $b = 1$

Implied: $c = 1, d = 1$

SAT!!

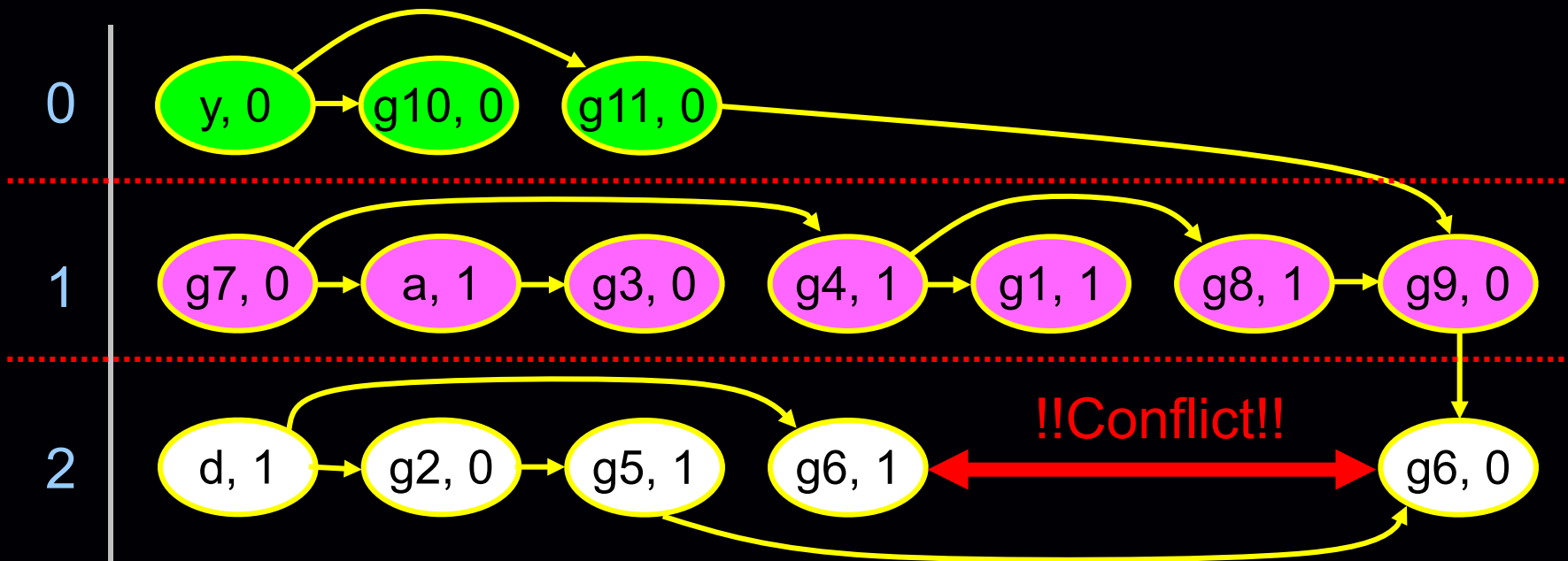
A Closer Look at the Implication Graph (a conceptual implementation)

- ◆ Implications are grouped into different decision levels
 - Level 0: target imp; constants
 - Level 1+: decisions
- ◆ Node (gate, value): implications
- ◆ Incoming edge(s) of a node: implication sources (reasons)
 - The nodes with no incoming edges are called “root implication nodes”
 - There should only be ONE root implication node for each decision level ≥ 1 (which is the decision in that level)

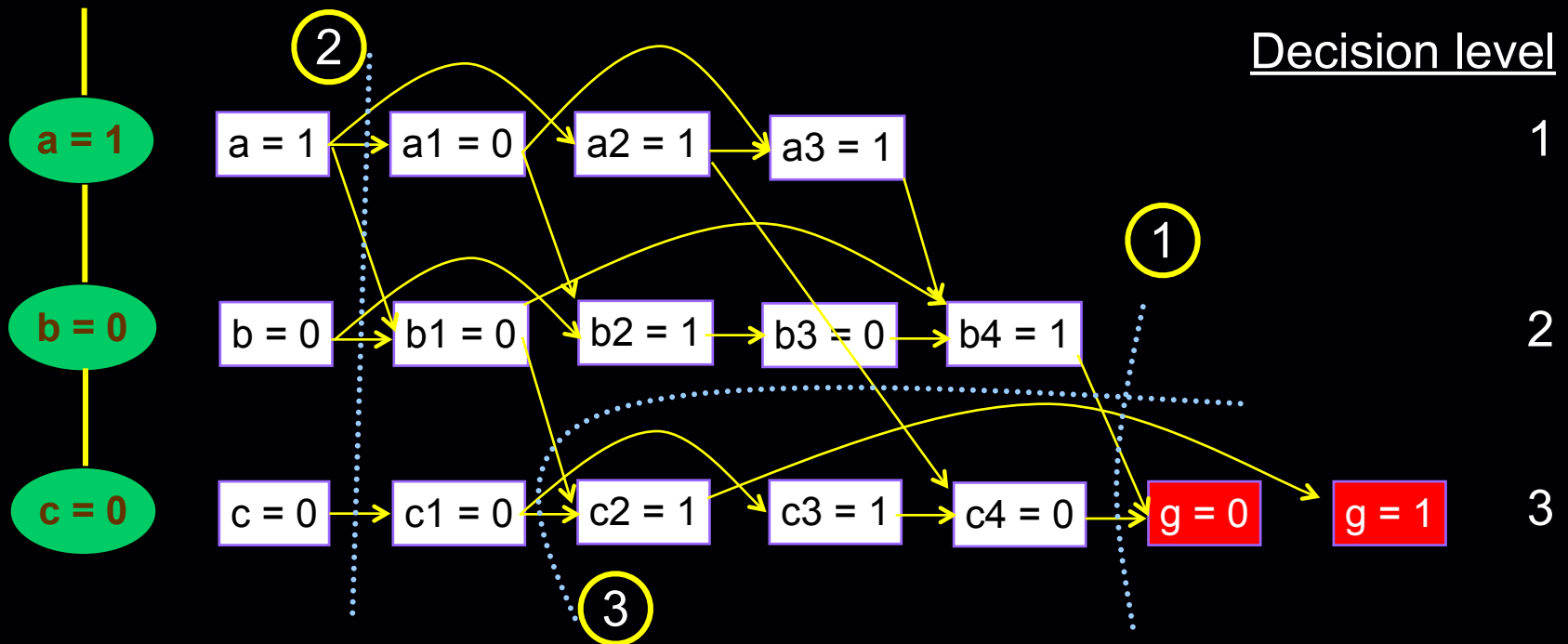


Conflict Analysis

- ◆ When we encounter a decision conflict, we want to figure out the causes so that ---
 1. Try to avoid the same conflict
 2. Backtrack as many decisions as possible

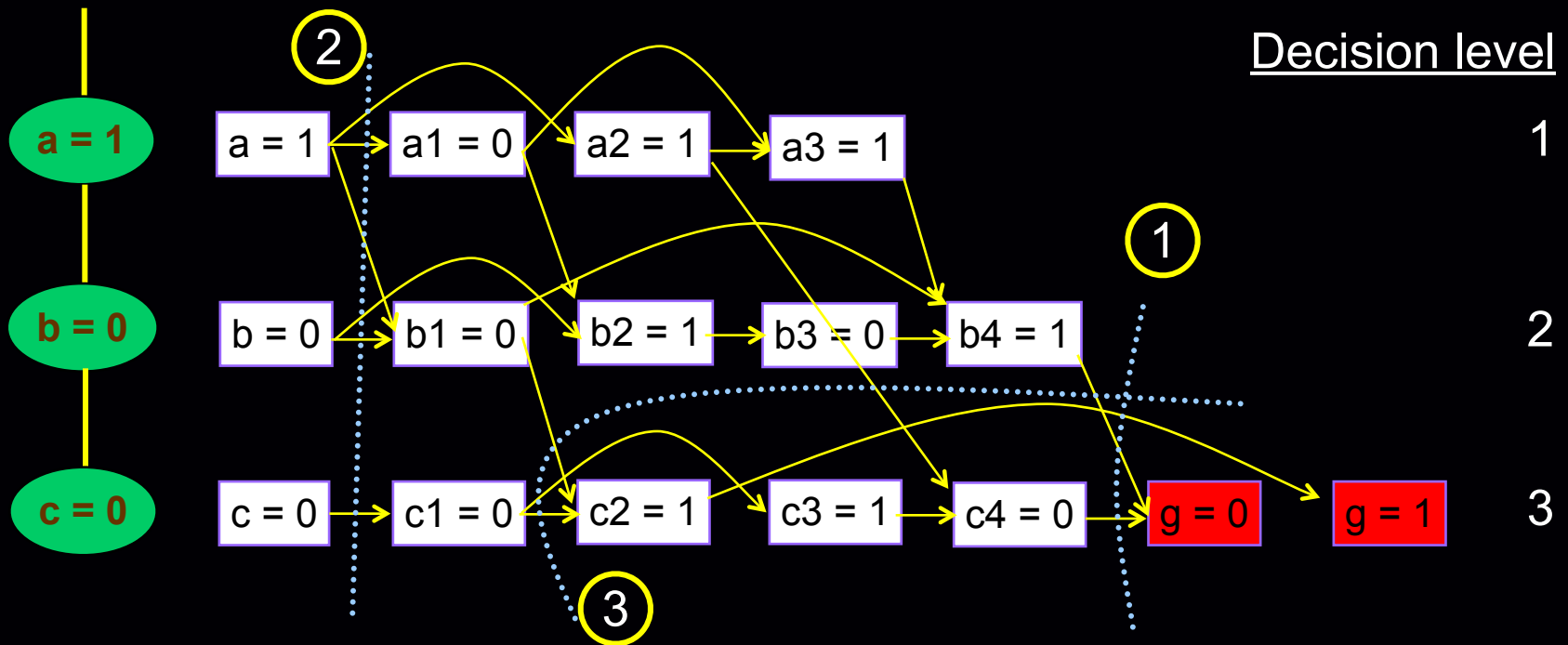


Conflict Analysis



1. Try to avoid the same conflict
 - Starting from the conflict implications ($g = 0$) & ($g = 1$), backward trace their implication sources
 - (An informal explanation) Any cut in the implication graph defines a set of conflict sources
 - Add a constraint for the conflict sources to prevent the conflict from happening again

Conflict-Driven Learning



- ◆ Add a constraint to prevent the same conflict
 1. $b4 \ \&\& \ c2 \ \&\& \ c4' = 0;$ $\rightarrow (b4' + c2' + c4)$
 2. $a \ \&\& \ b' \ \&\& \ c' = 0;$ $\rightarrow (a' + b + c)$
 3. $b4 \ \&\& \ a2 \ \&\& \ b1' \ \&\& \ c1' = 0;$ $\rightarrow (b4' + a2' + b1 + c1)$

Which constraint is the best to add?

- ◆ [Zhang, *et al*, ICCAD 2001] Experiment shows that “first-UIP” (1st-UIP) is the best
 - **UIP: Unique Implication Point**
 - In a cut that there is only one node (i.e. UIP) in the last (where conflict happens) decision level
(why UIP cut?)
 - Starting from the conflict gate, the first encountered UIP is namely first UIP
 - The cut with only decision nodes is the last-UIP
 - In the previous example, (2) is the last UIP, and (3) is the first UIP

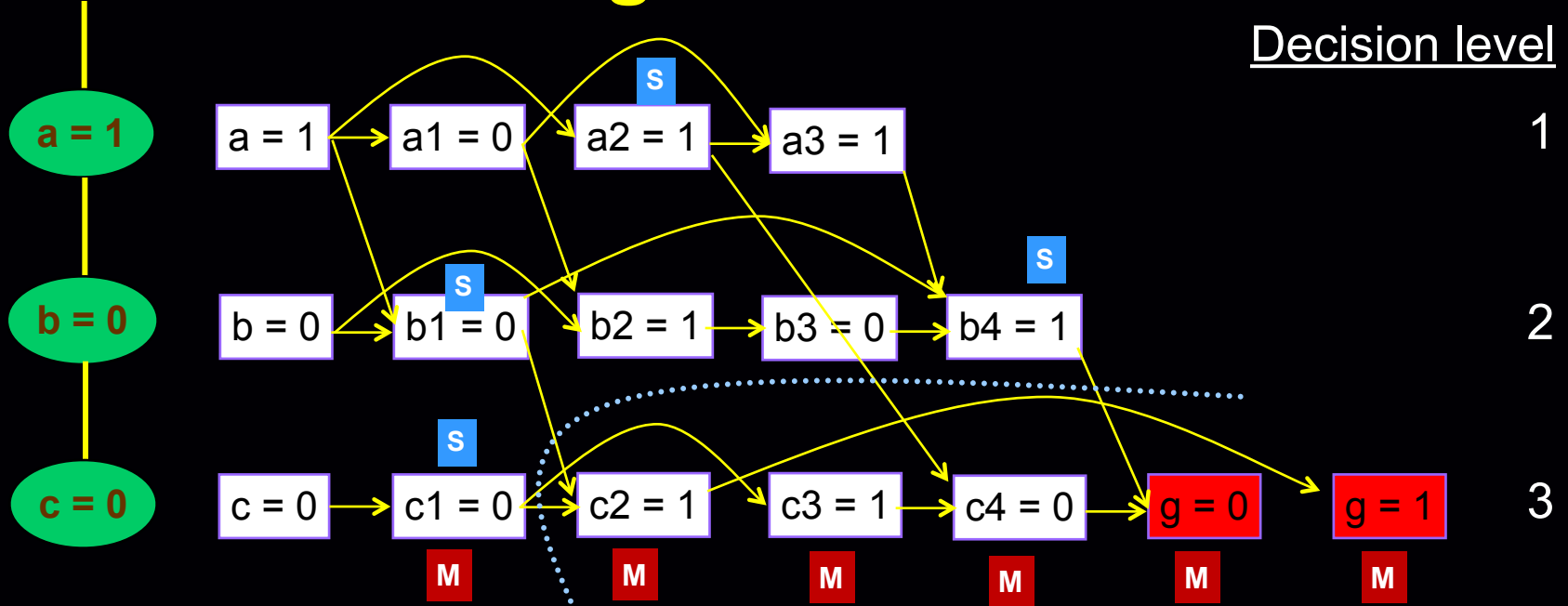
Pseudo-code to find the first UIP

```
conflictAnalysis(imp0Src, imp1Src) {
    int nMarked = 0;
    for_each_imp(imp, imp0Src)
        checkImp(imp, nMarked, conflictSrc);
    for_each_imp(imp, imp1Src)
        checkImp(imp, nMarked, conflictSrc);
    for_each_imp_rev(imp, lastDLevel) {
        if (!imp.isMarked()) continue;
        if (--numMarked == 0) { // UIP found!!
            conflictSrc.push_back(imp);
            break; // ready to return
        }
        imp.unsetMark();
    }
```

```
for_each_imp_src(imp_src, imp) {
    checkImp(imp_src, nMarked,
            conflictSrc);
}
}
for_each_imp(imp, conflictSrc)
    imp.unsetMark();
return conflictSrc;
}

checkImp(imp, nMarked, conflictSrc) {
    if (imp.isMarked()) return;
    imp.setMark();
    if (!imp.isLastDecisionLevel())
        conflictSrc.push_back(imp);
    else ++numMarked;
}
```

Linear-Time Algorithm to Find First UIP



- ◆ Start from $(g = 0)$, $(g = 1)$ // #Marks = 2
- ◆ Unmark $(g = 1)$, mark $(c_2 = 1)$ // #Marks = 2
- ◆ Unmark $(g = 0)$, mark $(c_4 = 0)$, add $(b_4 = 1)$ // #Marks = 2
- ◆ Unmark $(c_4 = 0)$, mark $(c_3 = 1)$, add $(a_2 = 1)$ // #Marks = 2
- ◆ Unmark $(c_3 = 1)$, mark $(c_1 = 0)$ // #Marks = 2
- ◆ Unmark $(c_2 = 1)$, add $(b_1 = 0)$ // #Marks = 1
- ◆ Find first UIP: $(c_1 = 0)$, conflict sources: $\{ (c_1 = 0), (b_1 = 0), (a_2 = 1), (b_4 = 1) \}$

UIP for Non-chronological Backtracking

- ◆ Since in UIP cut there is **only one node with the last decision level...**
- ◆ As we add a constraint for the UIP cut ---

Decision level

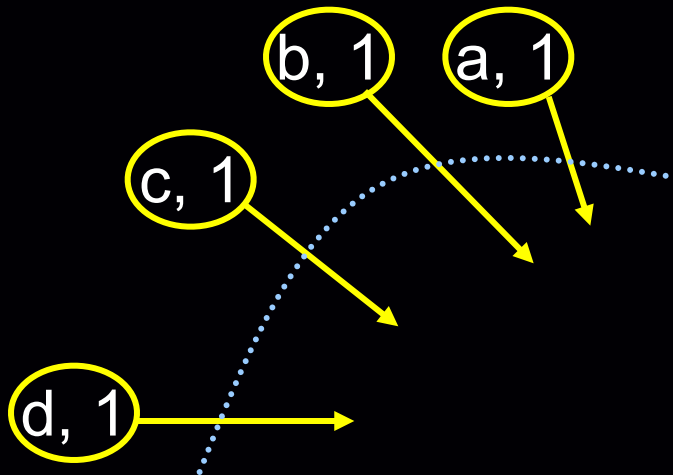
0

1

2

3

4



Constraint

$$(a \ \&\& \ b \ \&\& \ c \ \&\& \ d) = 0$$



If we backtrack to the second-to-the-last decision level (i.e. c)

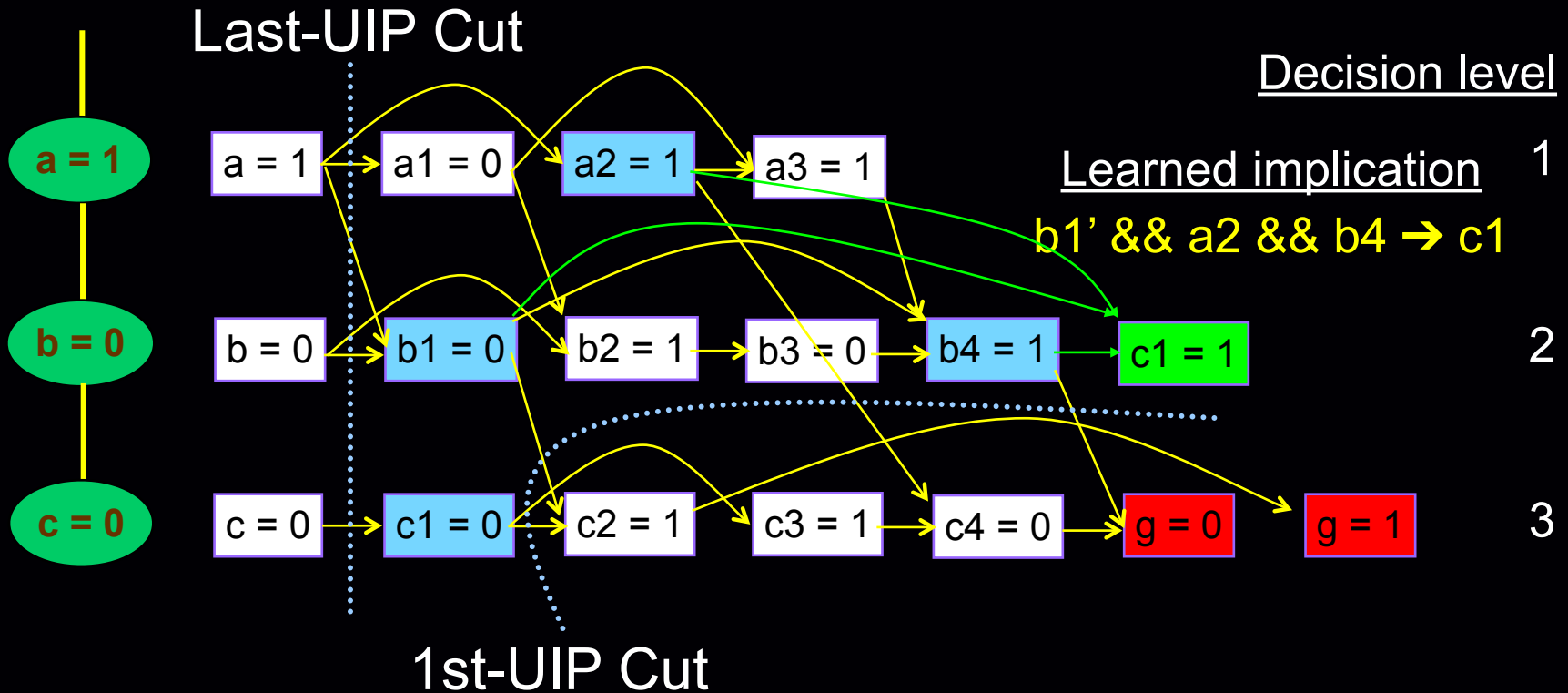
1. { a, b, c } still have the original implications

$$(a \ \&\& \ b \ \&\& \ c) \rightarrow d'$$

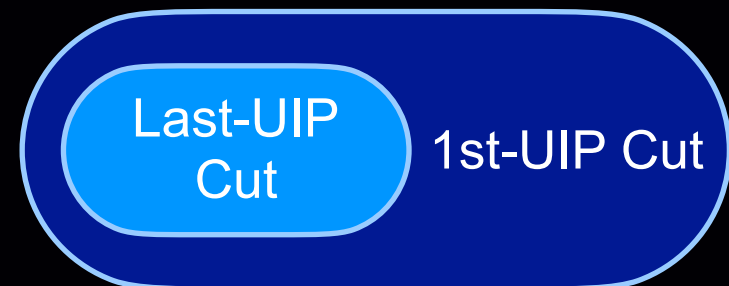


2. d can be implied with the opposite value at the max level above

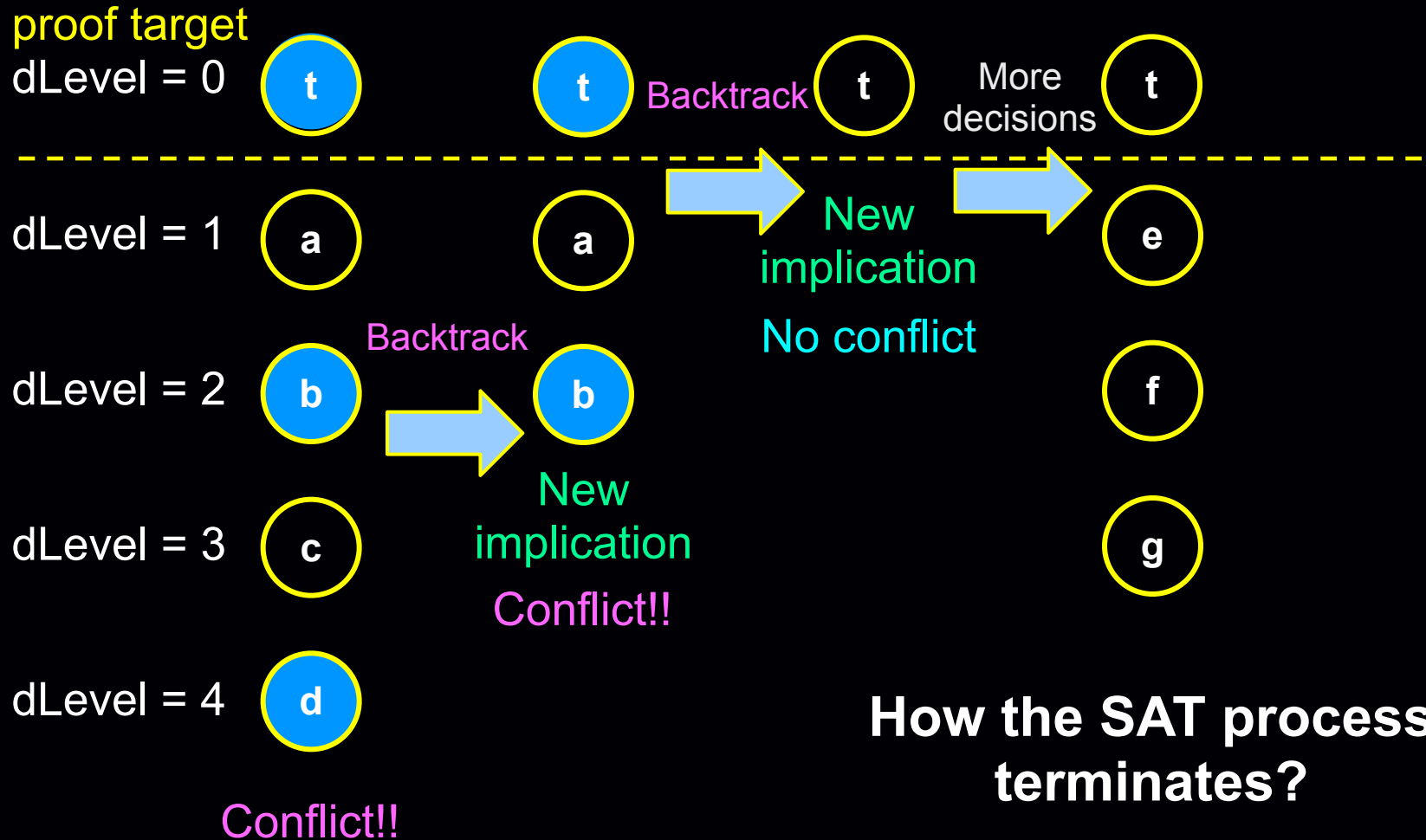
Why is first UIP the best cut?



Last-UIP Cut $\Rightarrow$ 1st-UIP Cut



Conflict-Driven Non-Chronological Backtracking --- Algorithm



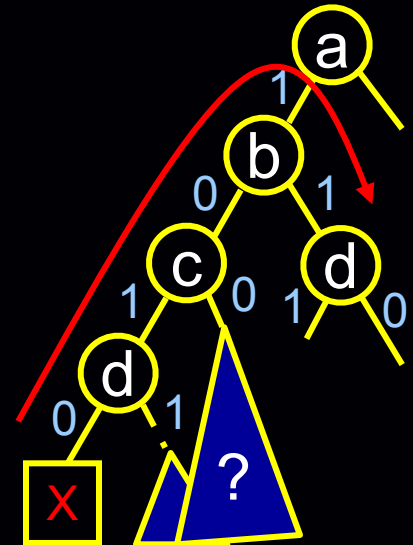
Conflict-Driven Non-Chronological Backtracking --- Algorithm

1. When conflict occurs, check if the conflict level == 0 (implication level for the SAT target)
 - a) If yes, return *unsatisfiability* (Why?)
 - b) Else, continue to 2
2. Find the 1st-UIP cut as the conflict causes
3. Backtrack to the max decision level of the nodes other than UIP in the cut
4. The UIP gate will be implied with the opposite value
5. Perform the new implication
6. If conflict, go to 1, else continue for the next decision

A closer look at binary decision tree

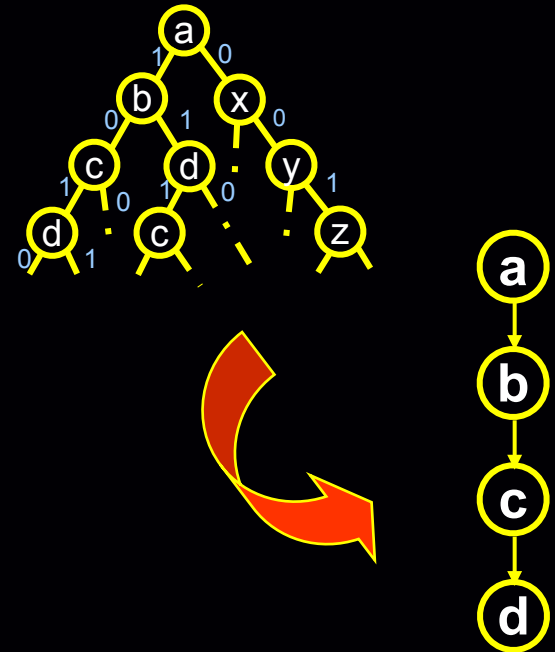
In general, is non-chronological backtracking safe?

- May lead to SAT solution earlier
- But some portion of the decision tree may not be covered
 - Not a complete search anymore
- May also miss some bugs
 - Difficult to record which branches haven't been searched



Conflict-Driven Non-Chronological Backtracking --- Completeness

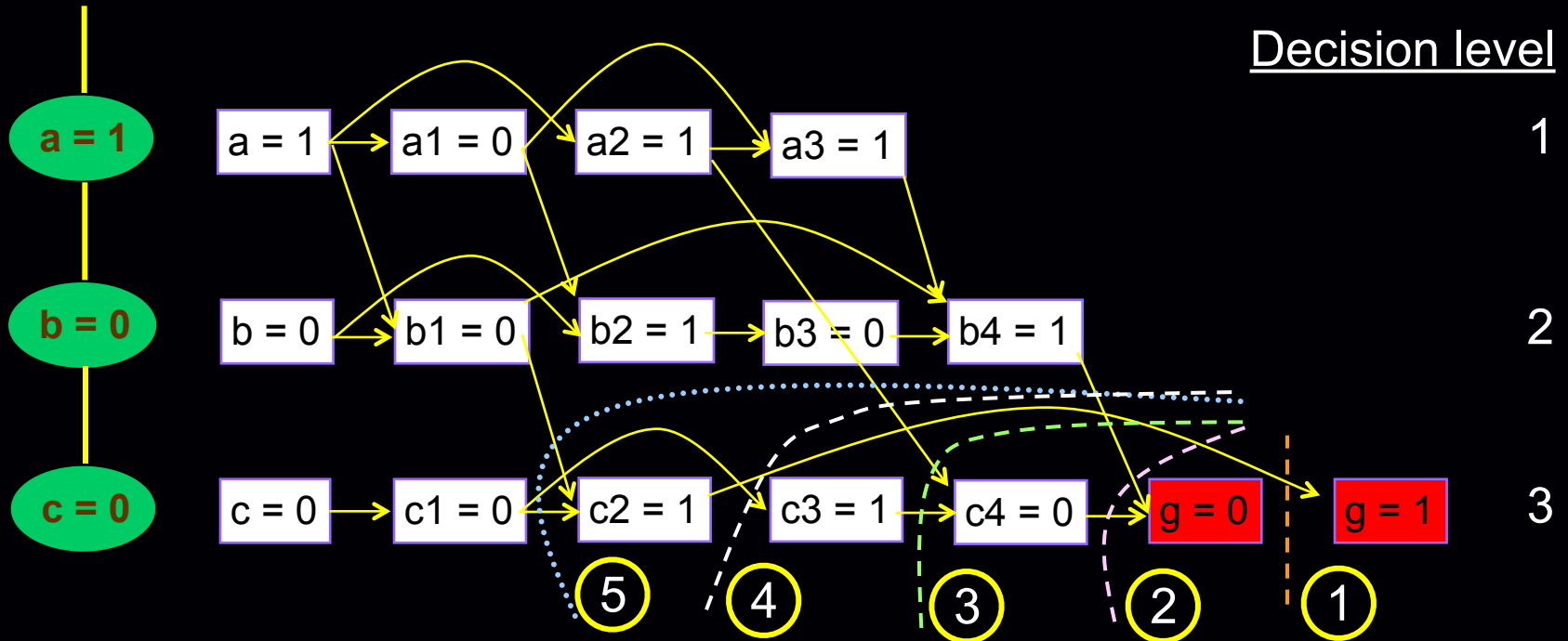
- ◆ But with conflict-driven learning, SAT search can be guaranteed to be complete
- ◆ SAT search is not a binary decision tree anymore...
 - Becomes a decision stack
 - Conflict
 - Learned clause (gate)
 - Indicate where to backtrack
 - Learned implication



Conflict-Driven Non-Chronological Backtracking --- Completeness

- ◆ Branch-and-bound algorithm for Constraint Satisfiability Problem (CSP) becomes a “constraint refinement process”
- Search region is gradually narrowed down
- At the end, either becomes empty, or finds the solution !!

Implication graph, resolution, and learning



(1): $(c2' + g)$

(2): $(b4' + c4 + g')$ $\rightarrow (b4' + c2' + c4)$

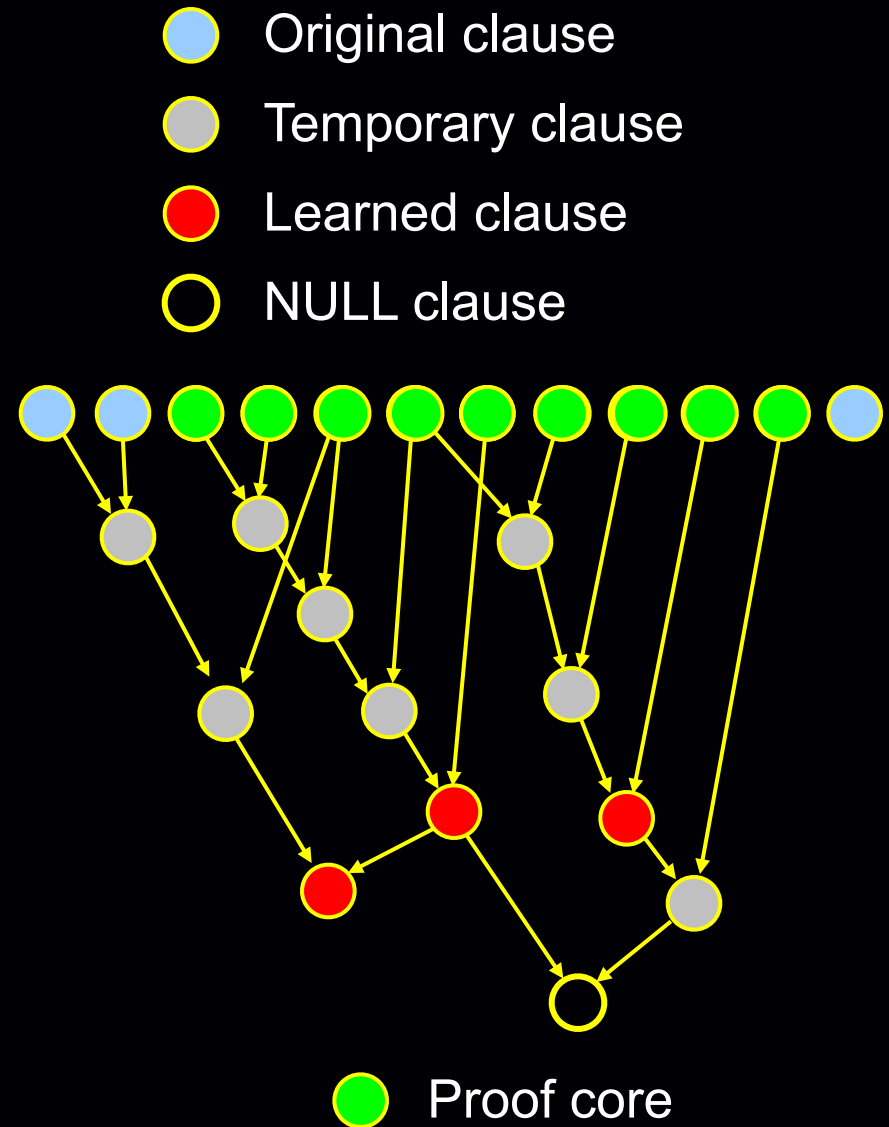
(3): $(a2' + c3' + c4')$ $\rightarrow (a2' + b4' + c2' + c3')$

(4): $(c1 + c3)$ $\rightarrow (a2' + b4' + c1 + c2')$

(5): $(b1 + c1 + c2)$ $\rightarrow (a2' + b1 + b4' + c1)$

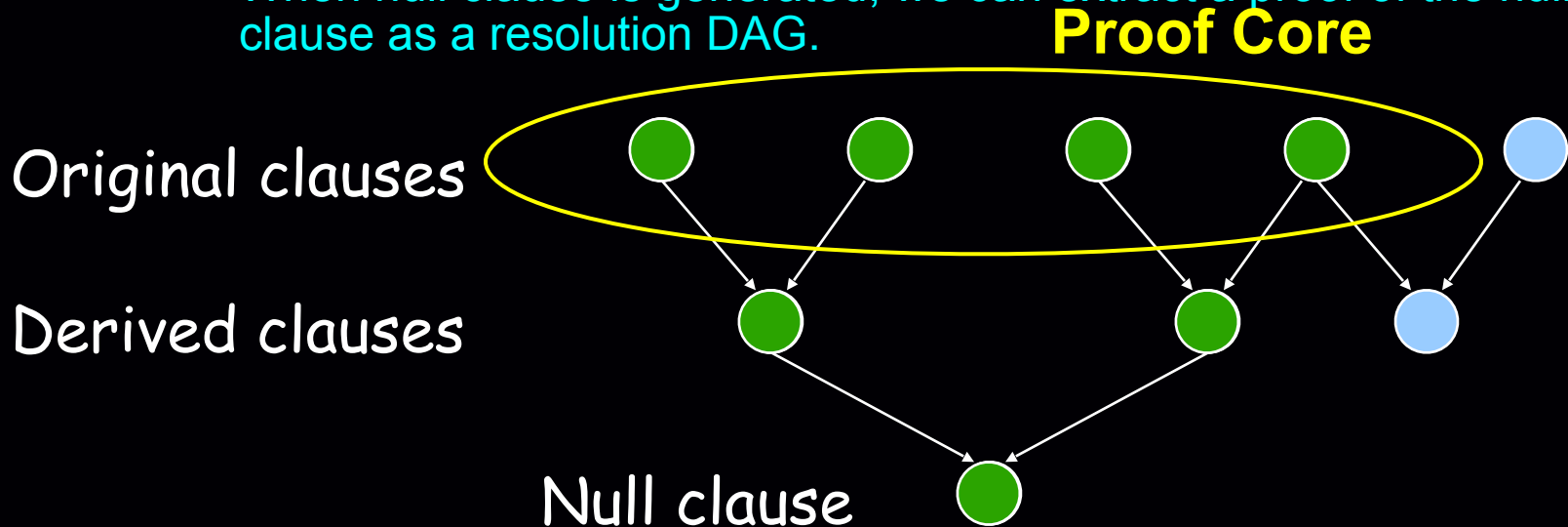
Resolution Graph

- ◆ A conflict is encountered
 - A learned clause is generated
- ◆ More conflicts are resolved...
- ◆ A conflict is encountered in decision level 0
 - Problem is proven UNSAT



Refutation / Proof Core of a SAT Problem

- ◆ Remember: Resolution-based SAT?
 - A problem is proven UNSAT if the resolution steps end up in a NULL clause
 - Tends to learn EVERYTHING!
- ◆ Refutation = a proof of the null clause
 - Also called “**proof core**” or “**UNSAT core**”
 - Record a DAG containing all resolution steps performed during conflict clause generation.
 - When null clause is generated, we can extract a proof of the null clause as a resolution DAG.



The validity of learned information and incremental SAT

- ◆ Note that, learned clause is a resolution of clauses that are involved in the implication process.
 - As long as these clauses are still in the proof database, the learned information is always valid.
- ◆ Incremental SAT
 - (For example) Proving two properties in a circuit --- the learned information obtained in proving one property can be reused in proving another.
 - (Challenge) What if some of the clauses or variables are deleted?